Resolução de Problemas
Estratégias de Busca

JOÃO LUÍS TAVARES DA SILVA

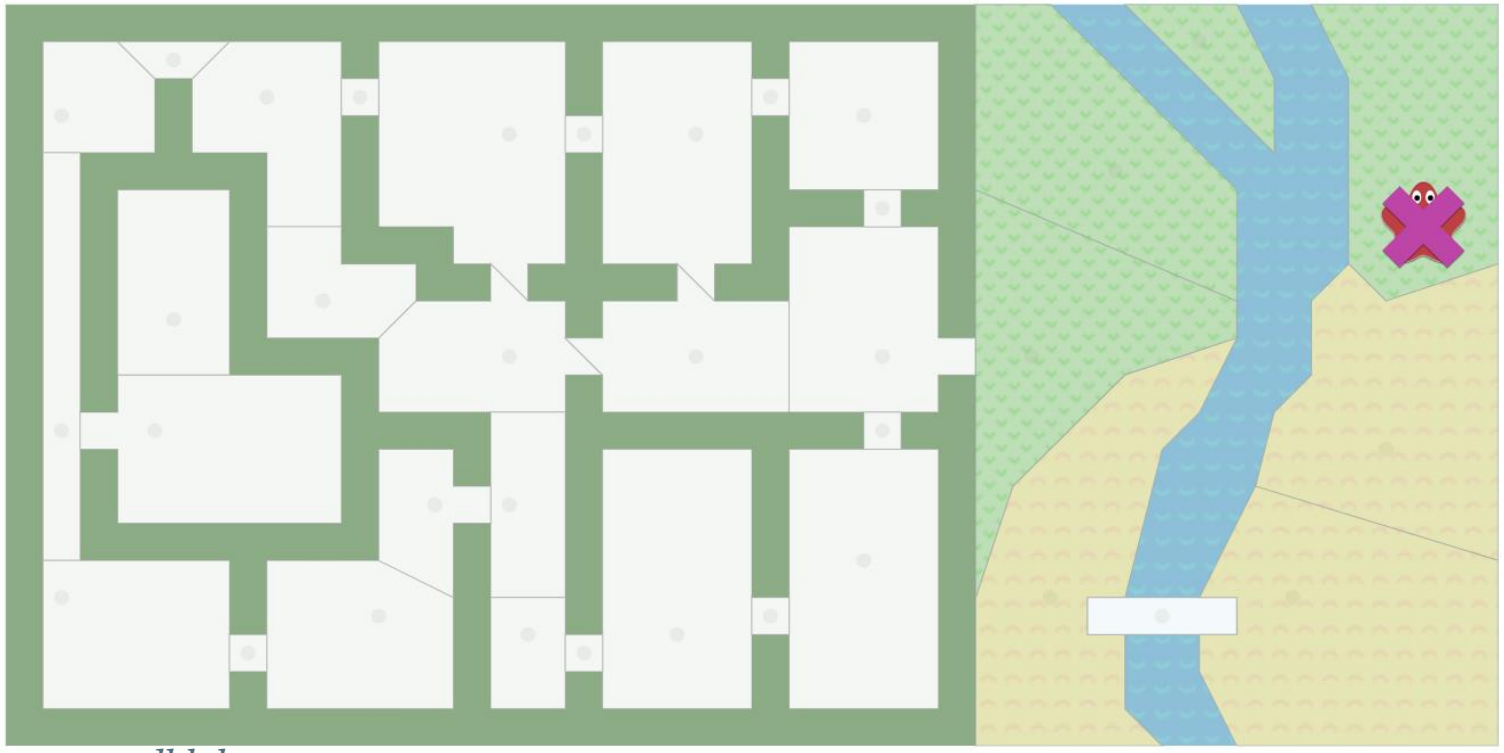
Problemas

- Problemas estão relacionados à resolução de tarefas humanas:
  - aquisição de informações(sentidos);
  - processamento de informações (raciocínio); e,
  - ações no meio exterior (capacidade de agir).

- Formalmente, um problema constitui-se em uma coleção de informações (conhecimento, regras, objetivos) que serão usadas para decidir o que fazer (solução).

Problema do Labirinto

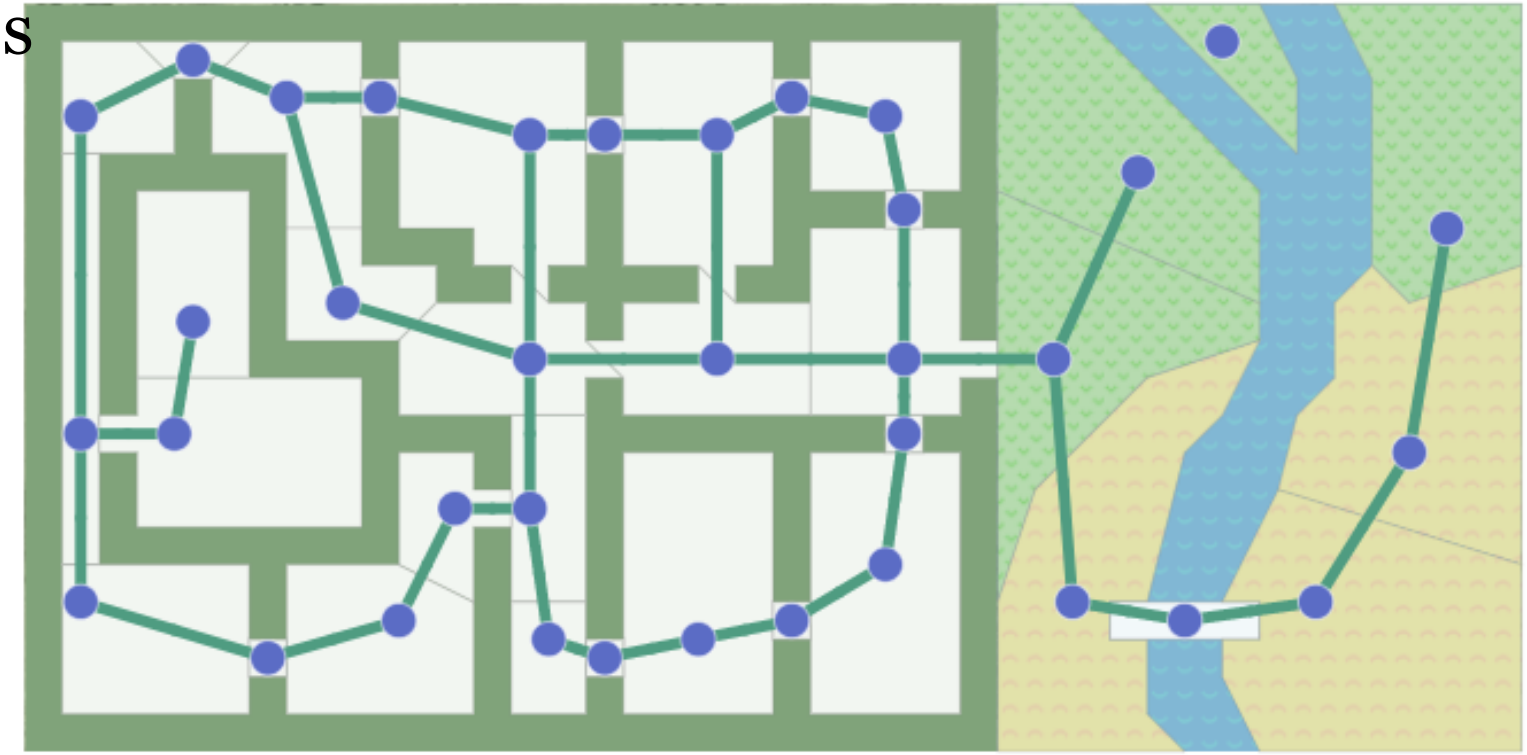
- *Pathfinder Problem*
- Encontrar o caminho mais curto em um labirinto



www.redblobgames.com

Problema do Labirinto

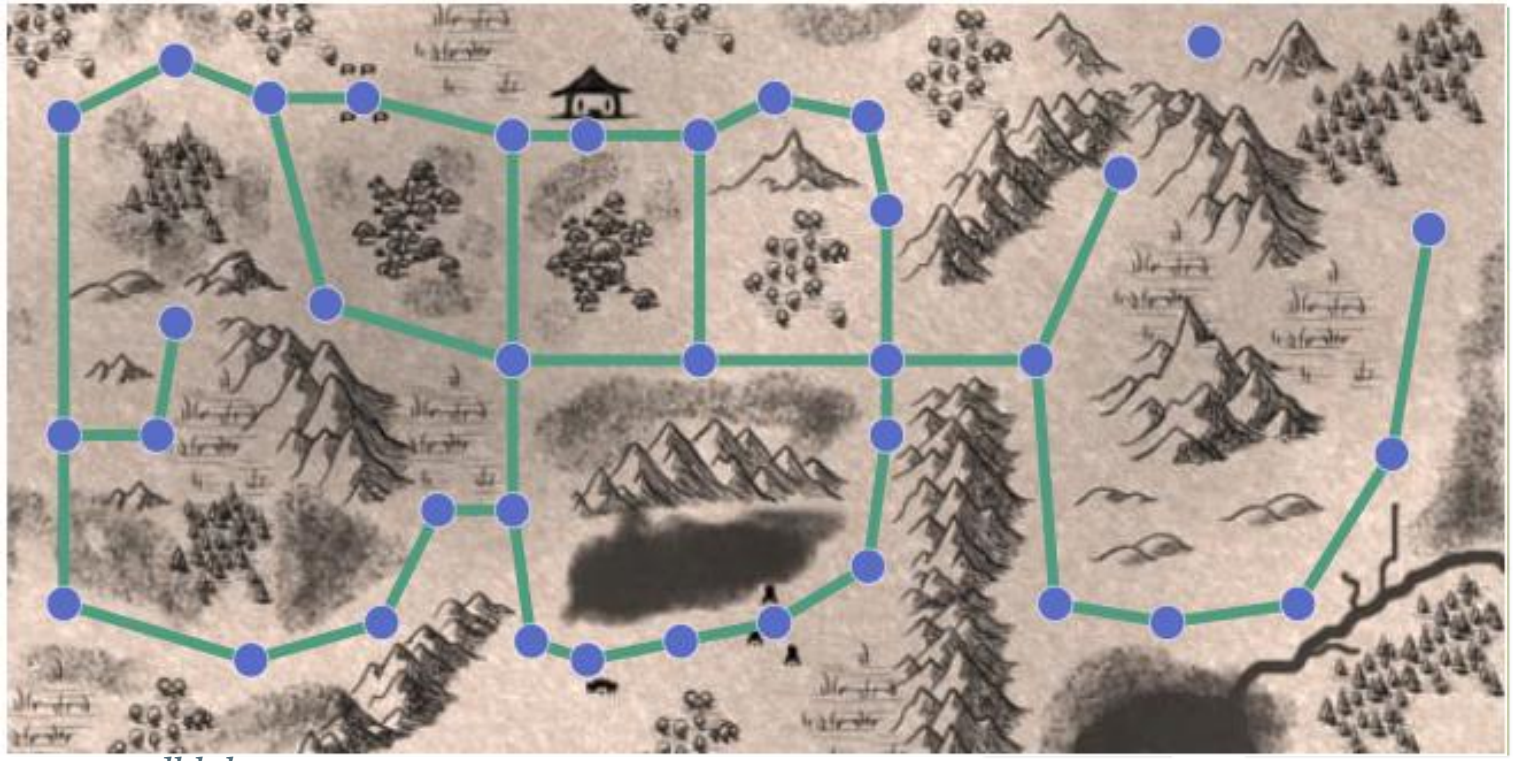
- *Pathfinder Problem*
- Encontrar o caminho mais curto em um labirinto
- Representação com grafos (direção, pesos)
- Vértices: conexões
- Arestas: caminhos



www.redblobgames.com

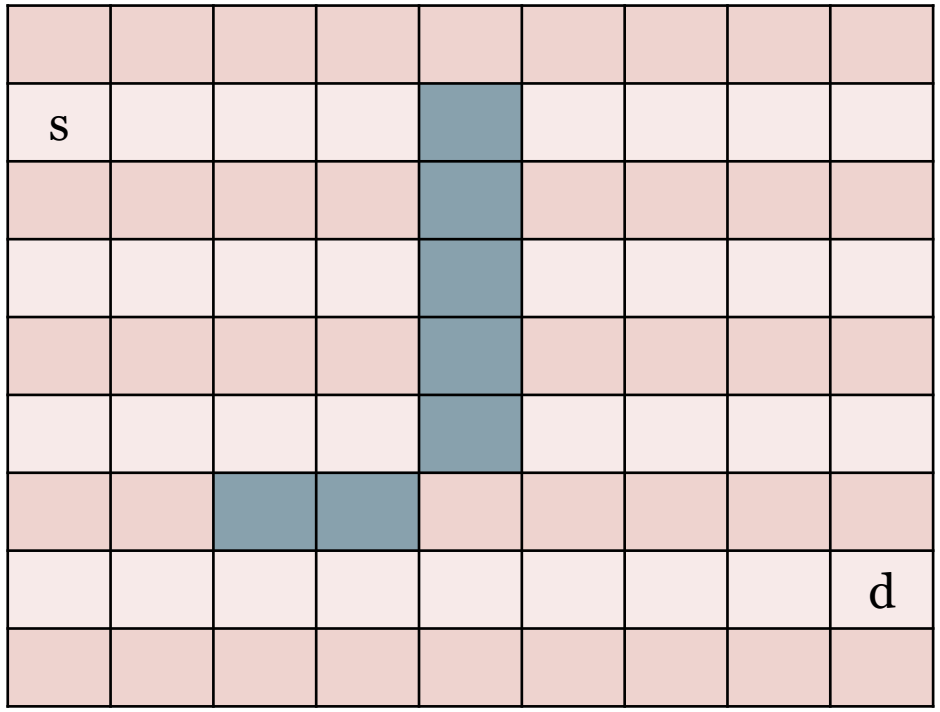
Problema do Labirinto

- *Pathfinder Problem*
- Pode-se abstrair para problemas similares...



www.redblobgames.com

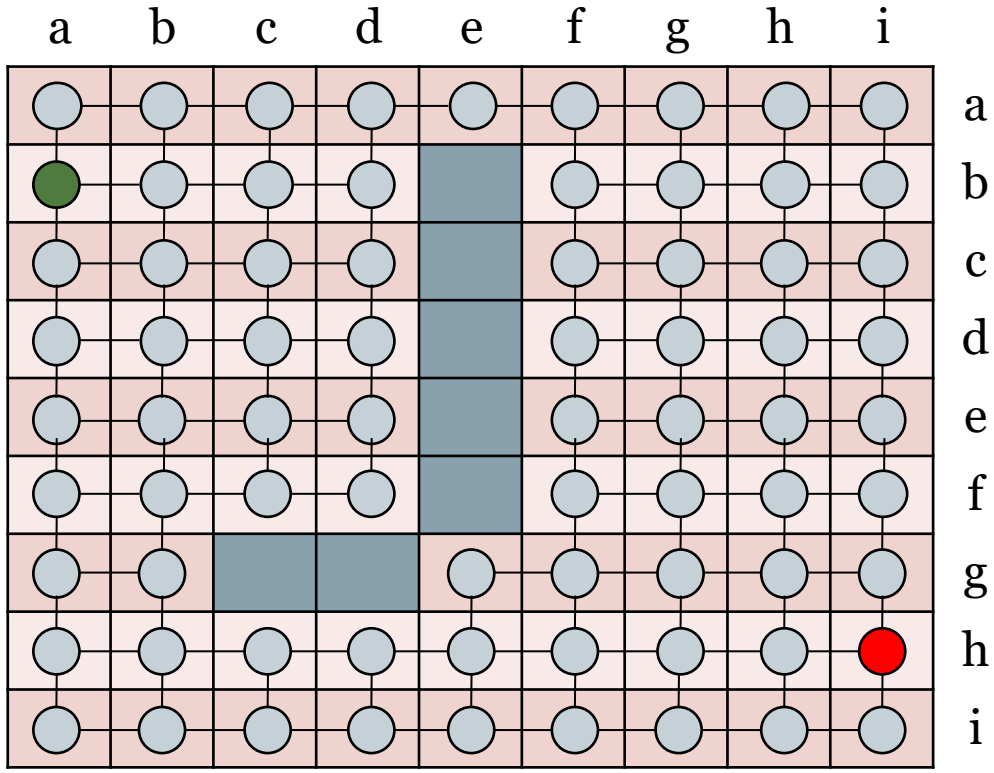
Grids e Grafos



- Encontrar o caminho mais curto em um labirinto
- Uma localização de início (*s*) e outra de destino (*d*)
- Eventualmente obstáculos

Como representar o problema?

Grids e Grafos



- Cada posição no grid se torna um vértice do grafo
- Arestas entre dois vértices posições são vizinhas
- Obstáculos desconectam os vértices, por exemplo:
  - *(b,d)* não é vizinho de *(b,f)*

Estratégias de Busca

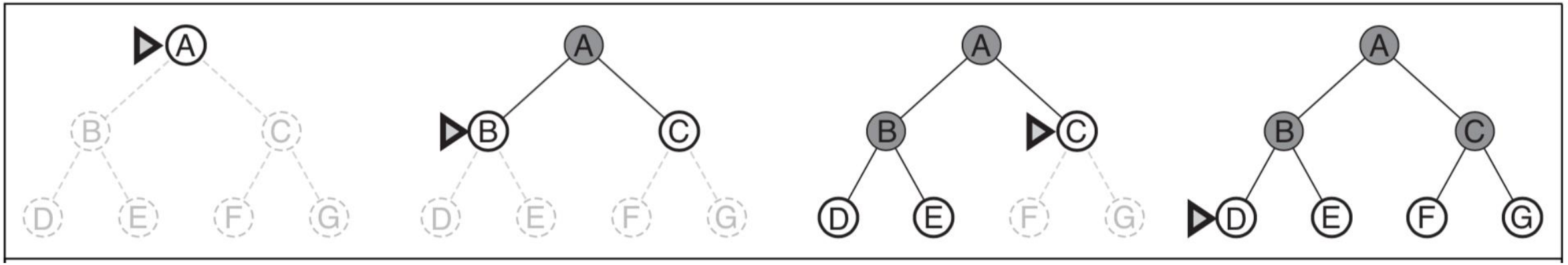
- Busca “cega” ou não informada
  - Amplitude e Amplitude com custo uniforme
  - Profundidade, profundidade limitada e profundidade iterativa
  - Busca bi-direcional

- Busca Heurística ou Informada(*Best-First*)
  - Busca “gulosa” (*greedy*)
  - Busca A*
  - Busca IDA (A* interativo)
  - Subida de encosta (*HillClimbing*)

Busca em Amplitude

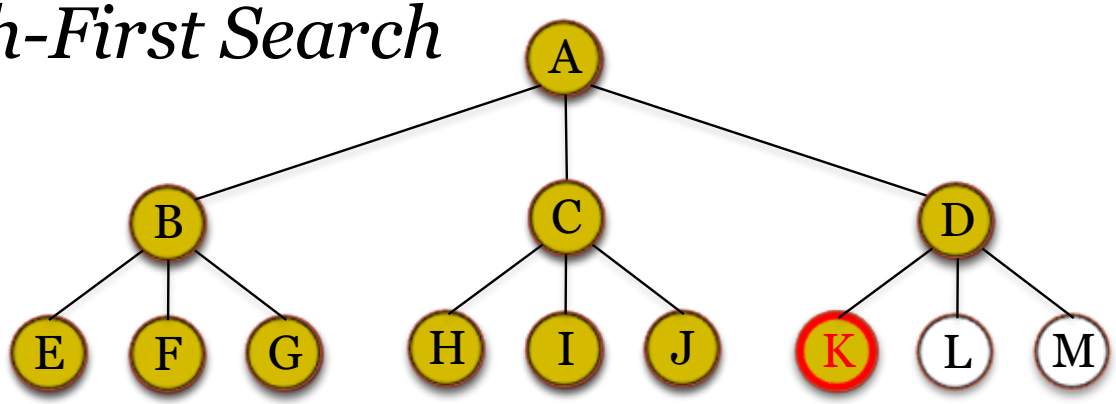
- Busca em largura, amplitude ou *Breath-first-Search (BFS)*

- O grafo da solução é construído em níveis:
  - a partir do nó raiz;
  - expande-se todos os sucessores de cada nível.



Busca Amplitude - Algoritmo

- *Breath-First Search*



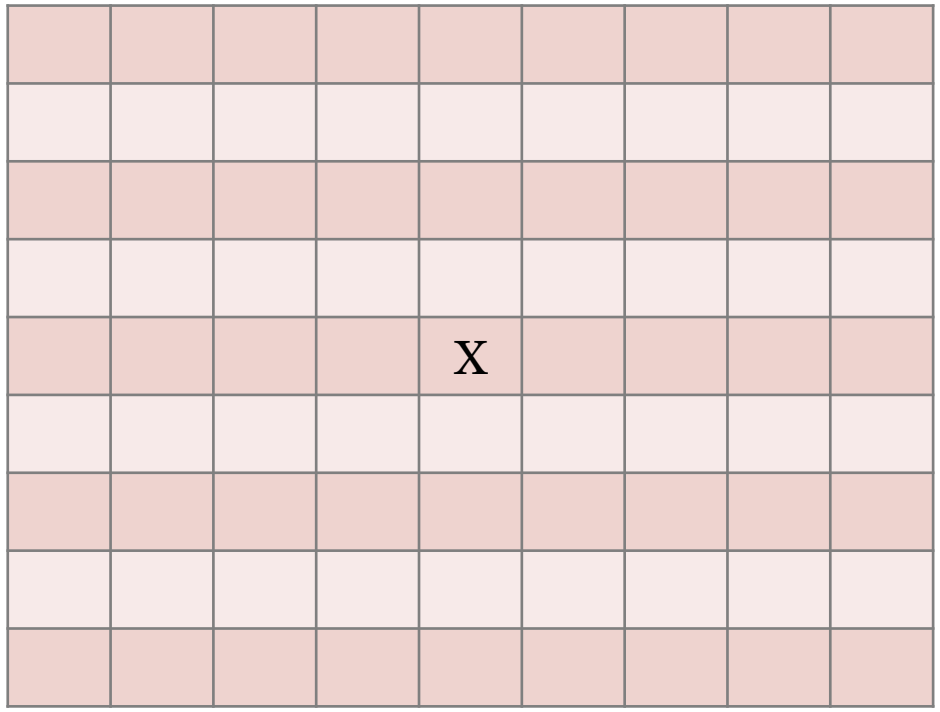
BFS(estadoInicial, sucesso)

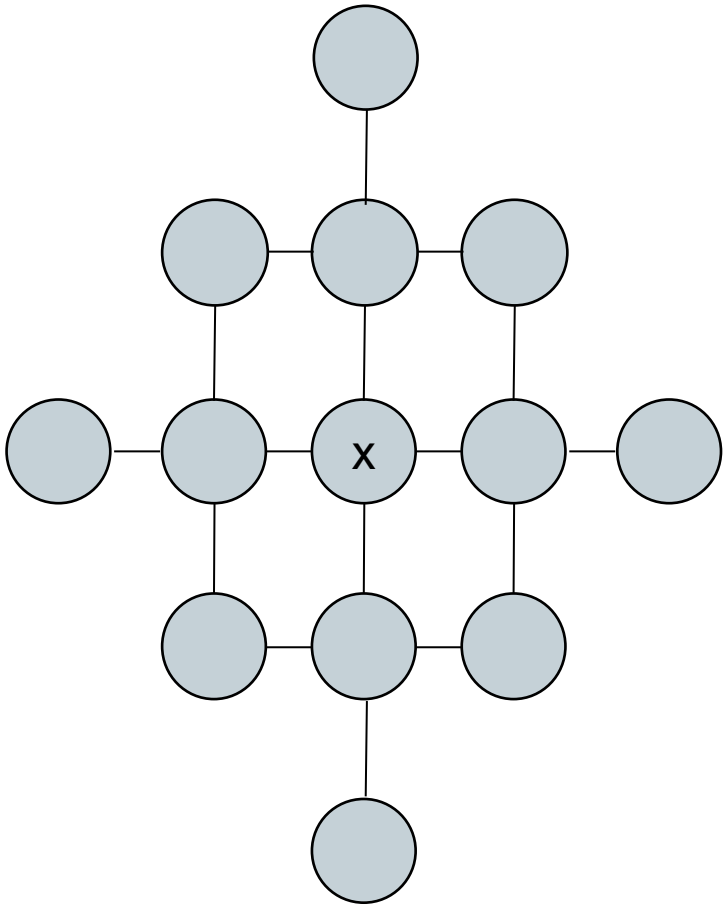
nodos ← MAKE-QUEUE(estadoInicial)
REPITA
  SE nodos é vazio **ENTÃO RETORNA** falha
  nodo ← REMOVE-FIRST(nodos)
  SE GOAL-TEST(nodo)=sucesso **ENTÃO RETORNA** nodo
  novos-nodos ← EXPAND(nodo)
  nodos ← ADD-**END**-QUEUE(nodos, novos-nodos)
FIM

FIFO (First In, First Out)

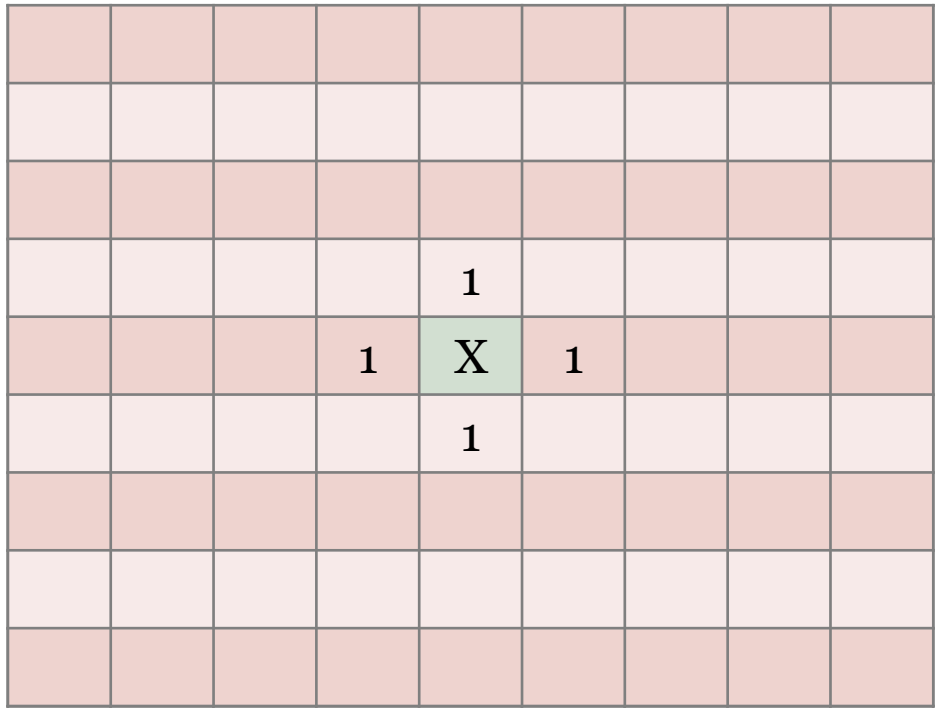
1. F={**A**}
2. F={**B** C D}
3. F={**C** D E F G}
4. F={**D** E F G H I J}
5. F={**E** F G H I J K L M}
6. F={**F** G H I J K L M}
7. F={**G** H I J K L M}
8. F={**H** I J K L M}
9. F={**I** J K L M}
10. F={**J** K L M}
11. F={**K** L M}
12. Sucesso K

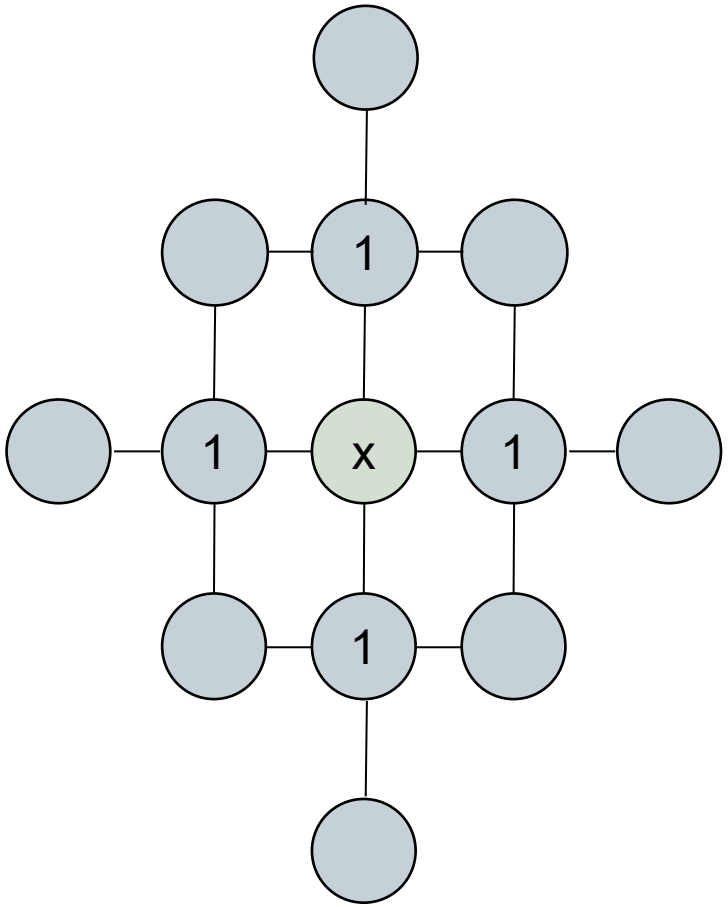
Como percorrer o Labirinto?



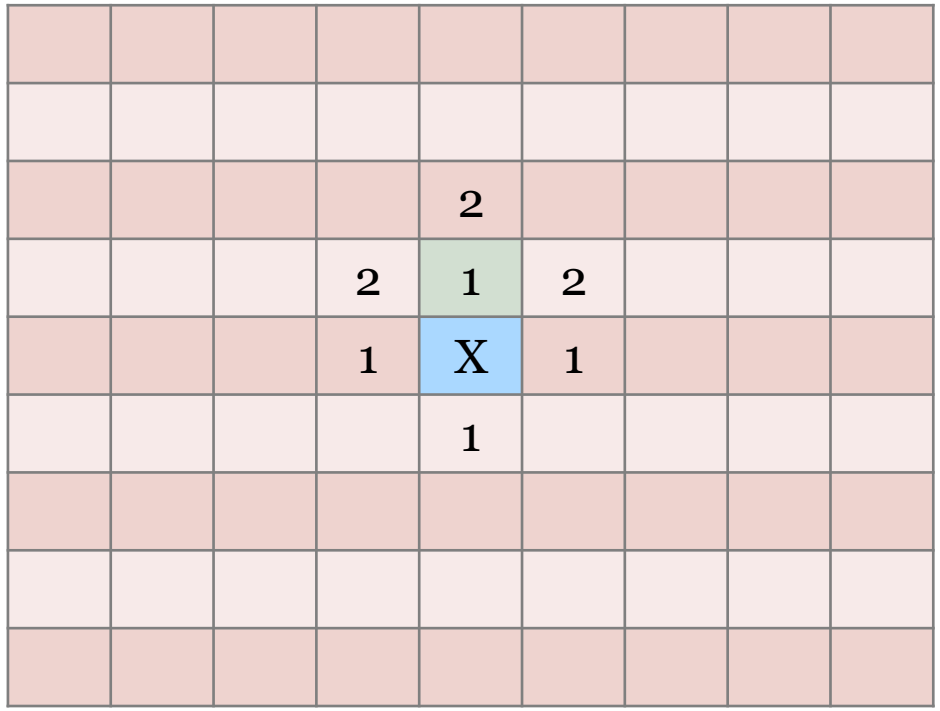


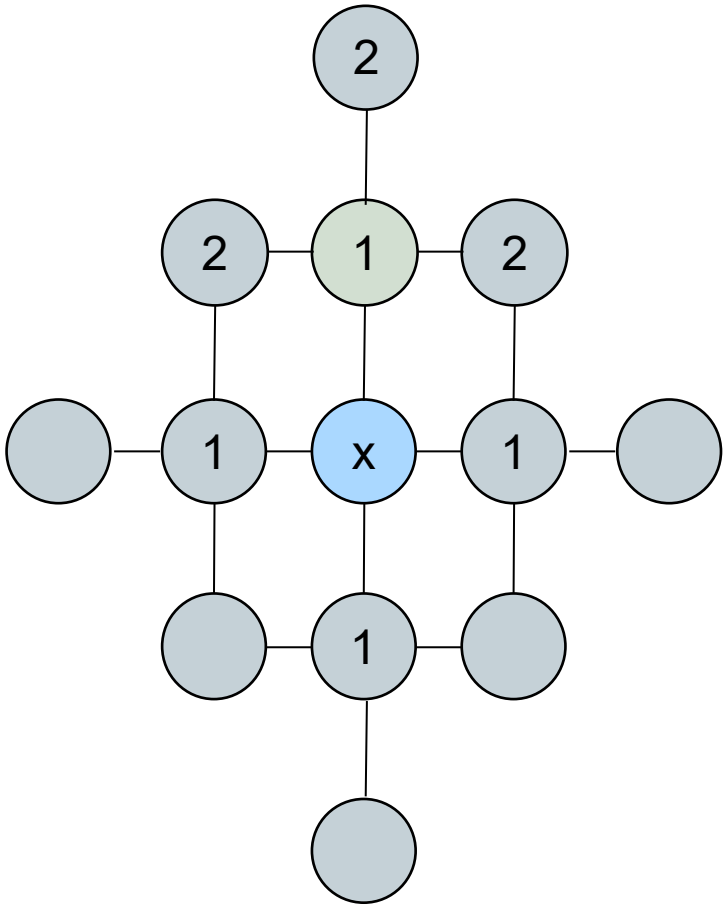
Busca em Largura



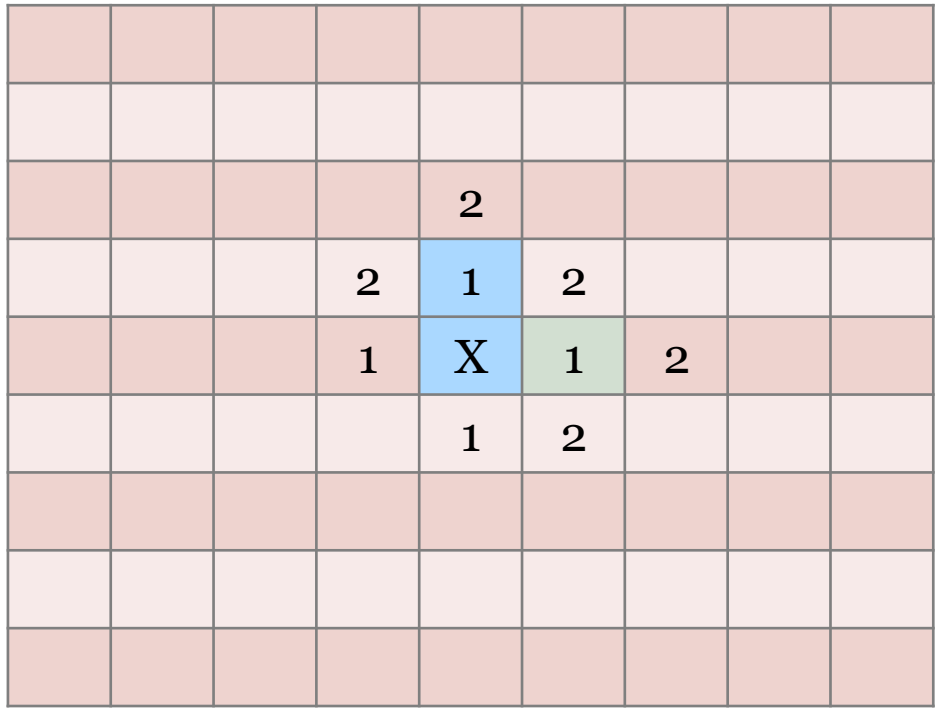


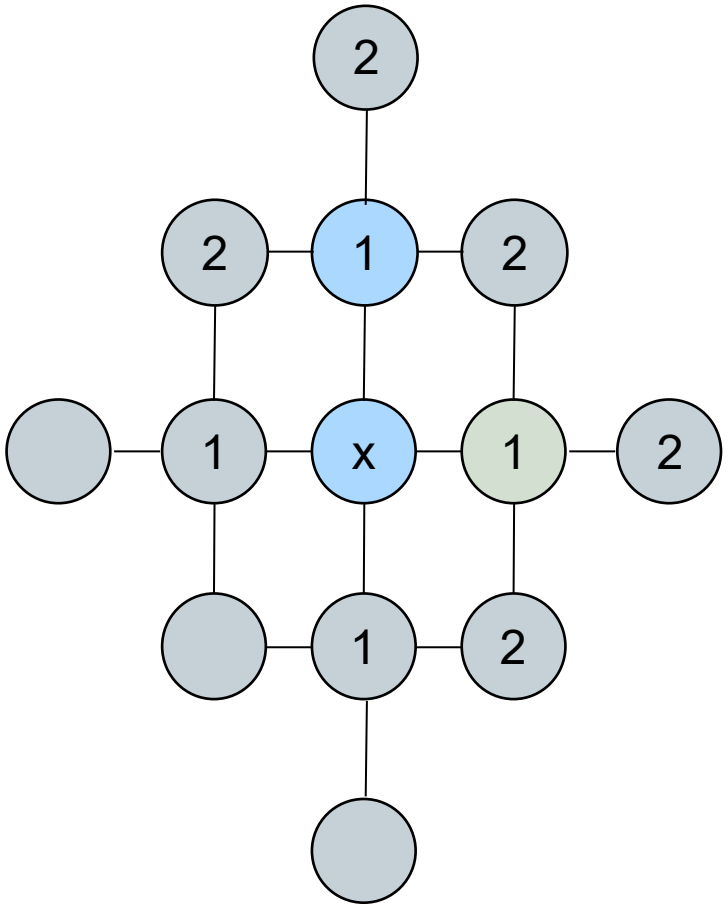
Busca em Largura





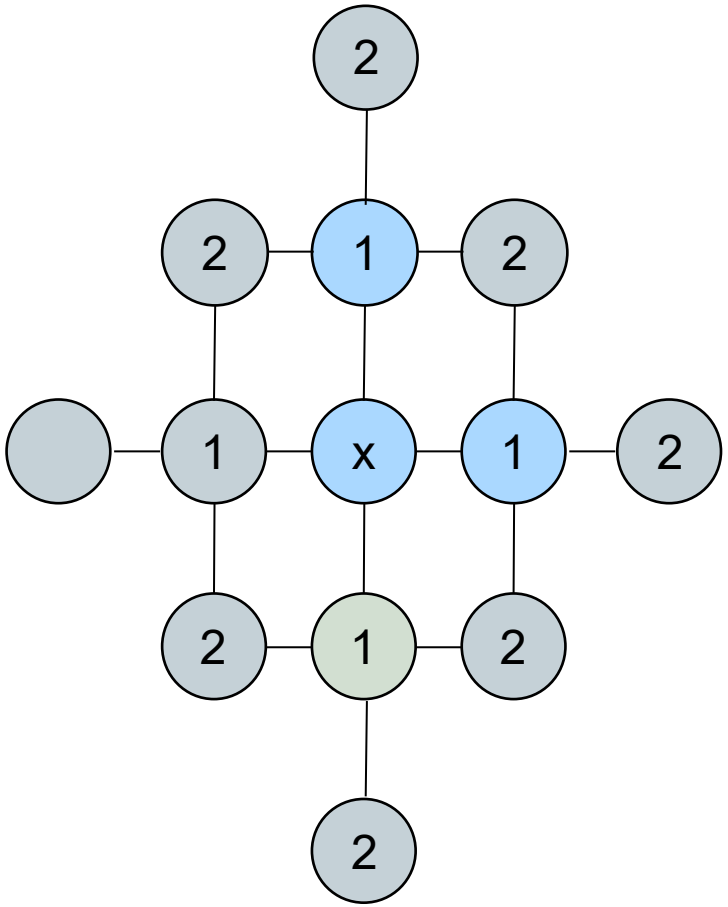
Busca em Largura



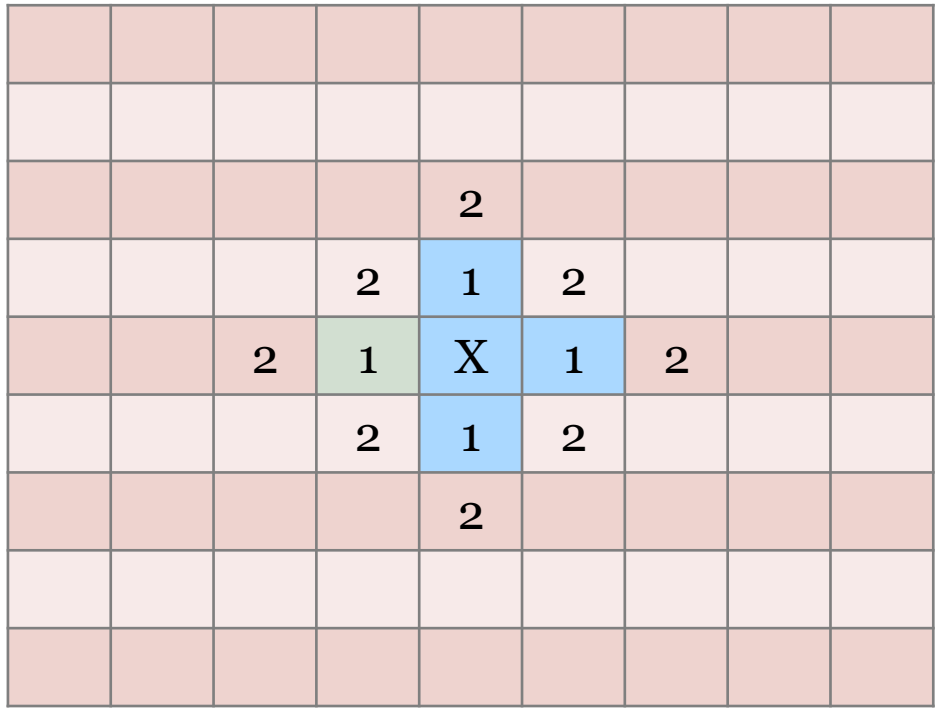


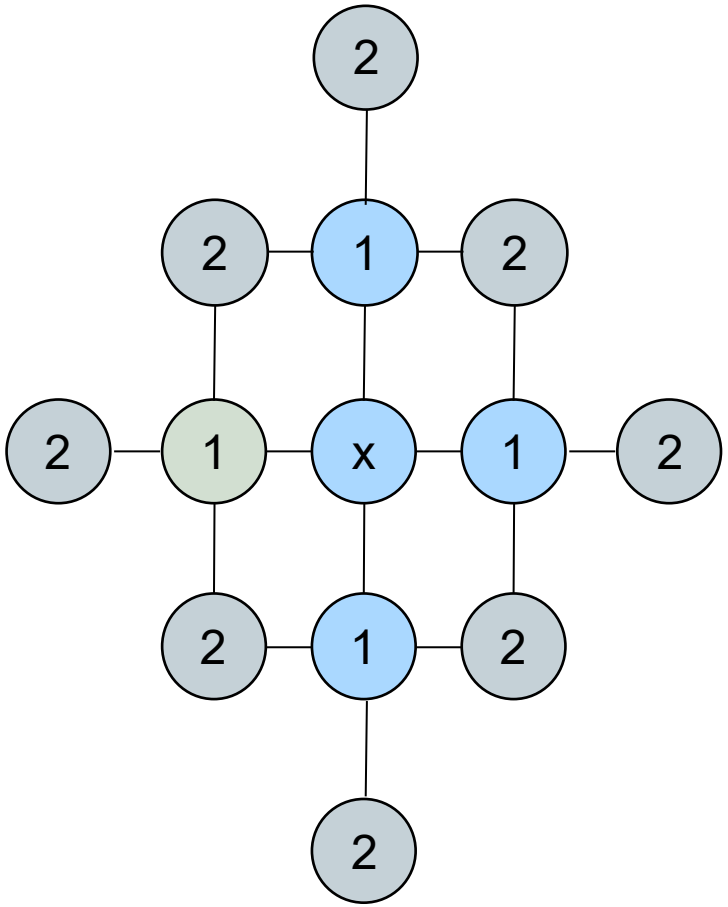
Busca em Largura

				2				
			2	1	2			
			1	X	1	2		
			2	1	2			
				2				



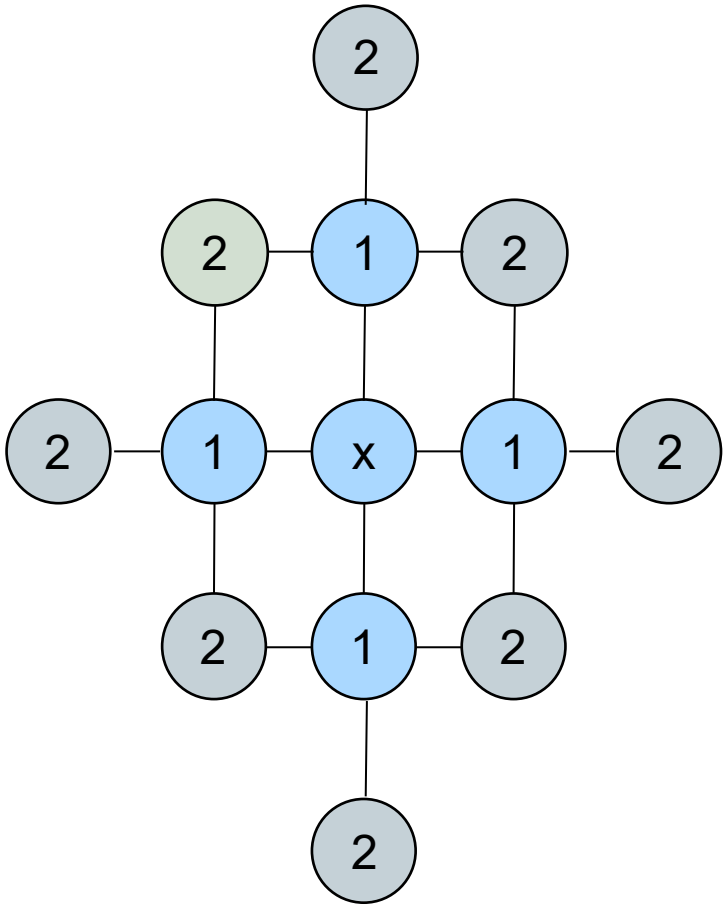
Busca em Largura





Busca em Largura

			3	2				
		3	2	1	2			
		2	1	X	1	2		
			2	1	2			
				2				



Busca em Largura

				3				
			3	2	3			
		3	2	1	2			
		2	1	X	1	2		
			2	1	2			
				2				

Busca em Largura

				3				
			3	2	3			
		3	2	1	2	3		
		2	1	X	1	2		
			2	1	2			
				2				

Busca em Largura

				3				
			3	2	3			
		3	2	1	2	3		
		2	1	X	1	2	3	
			2	1	2	3		
				2				

Busca em Largura

				3				
			3	2	3			
		3	2	1	2	3		
		2	1	X	1	2	3	
			2	1	2	3		
				2	3			

Busca em Largura

				3				
			3	2	3			
		3	2	1	2	3		
		2	1	X	1	2	3	
			2	1	2	3		
			3	2	3			
				3				

Busca em Largura

				3				
			3	2	3			
		3	2	1	2	3		
		2	1	X	1	2	3	
		3	2	1	2	3		
			3	2	3			
				3				

Busca em Largura

				3				
			3	2	3			
		3	2	1	2	3		
	3	2	1	X	1	2	3	
		3	2	1	2	3		
			3	2	3			
				3				

Busca em Largura

				3				
		4	3	2	3			
	4	3	2	1	2	3		
	3	2	1	X	1	2	3	
		3	2	1	2	3		
			3	2	3			
				3				

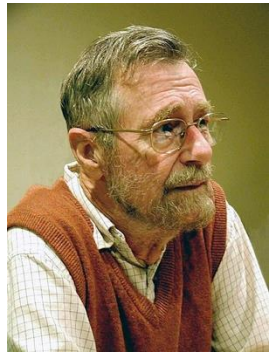
Busca em Largura

8	7	6	5	4	5	6	7	8
7	6	5	4	3	4	5	6	7
6	5	4	3	2	3	4	5	6
5	4	3	2	1	2	3	4	5
4	3	2	1	X	1	2	3	4
5	4	3	2	1	2	3	4	5
6	5	4	3	2	3	4	5	6
7	6	5	4	3	4	5	6	7
8	7	6	5	4	5	6	7	8

Busca com Custo Uniforme

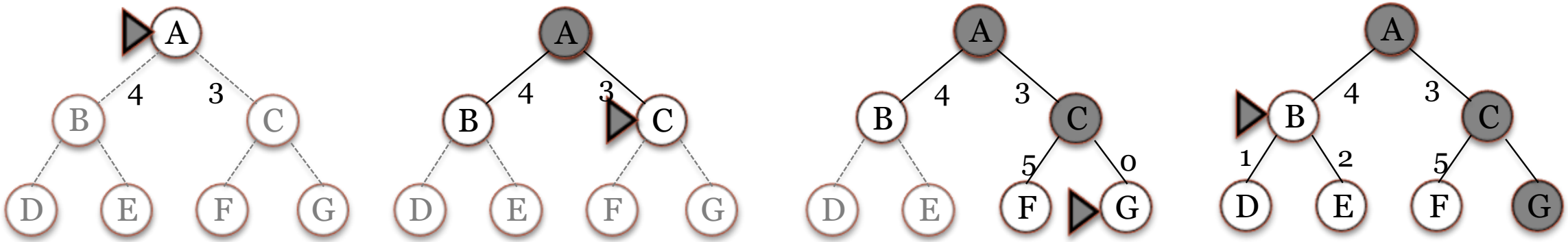
- Variação da Busca em Amplitude, onde o nodo com o menor custo é expandido primeiro
- Possue a mesma análise de critérios da Busca em Amplitude

Algoritmos de Dijkstra ou Bellman-Ford



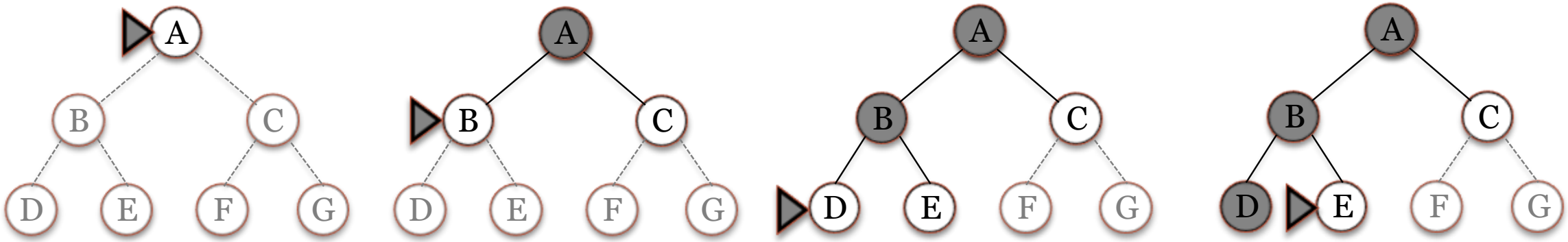
Edsger W. Dijkstra (1956)

Richard E. Bellman (1958)



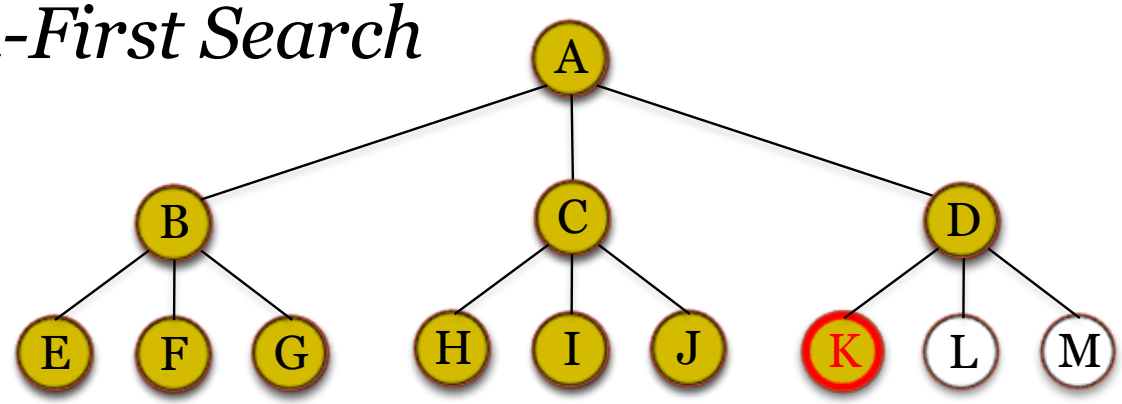
Busca em Profundidade

- Busca em profundidade ou *Depth-first-Search (DFS)*
- O grafo da solução é construída em profundidade:
  - a partir do nó raiz, é expandido sempre o filho mais à esquerda até a profundidade máxima



Busca Profundidade - Algoritmo

- *Depth-First Search*



DFS (estadoInicial, sucesso)

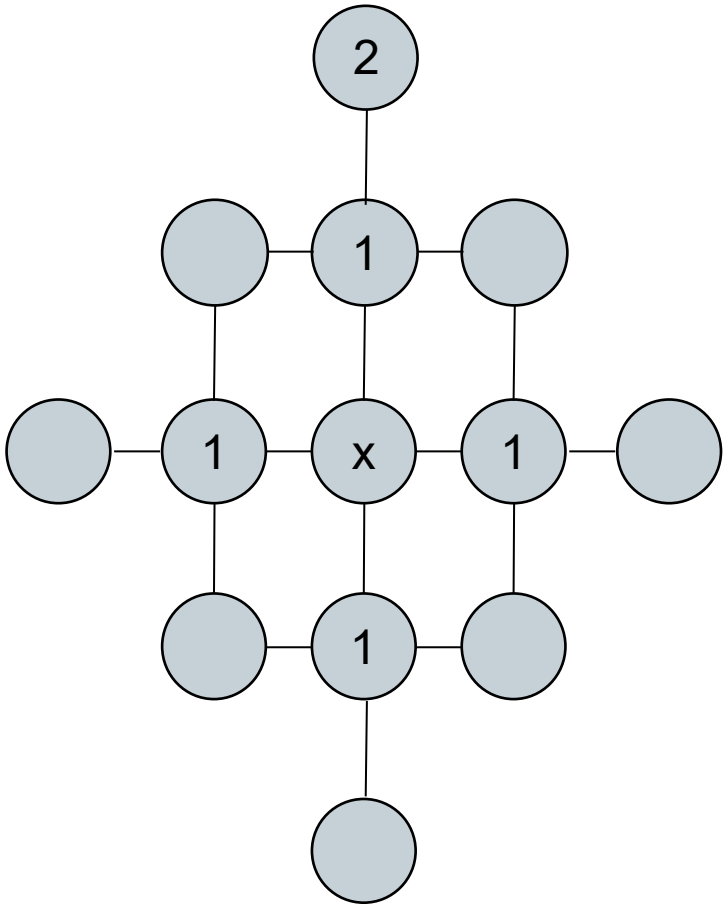
nodos ← MAKE-QUEUE(estadoInicial)
REPITA
  SE nodos é vazio **ENTÃO RETORNA** falha
  nodo ← REMOVE-FIRST(nodos)
  SE GOAL-TEST(nodo)=sucesso **ENTÃO RETORNA** nodo
  novos-nodos ← EXPAND(nodo)
  nodos ← ADD-**START**-QUEUE(nodos, novos-nodos)
FIM

LIFO (Last In, First Out)

1. F={**A**}
2. F={**B** C D}
3. F={**E** F G C D}
4. F={**F** G C D}
5. F={**G** C D}
6. F={**C** D}
7. F={**H** I J D}
8. F={**I** J D}
9. F={**J** D}
10. F={**D**}
11. F={**K** L M}
12. Sucesso K

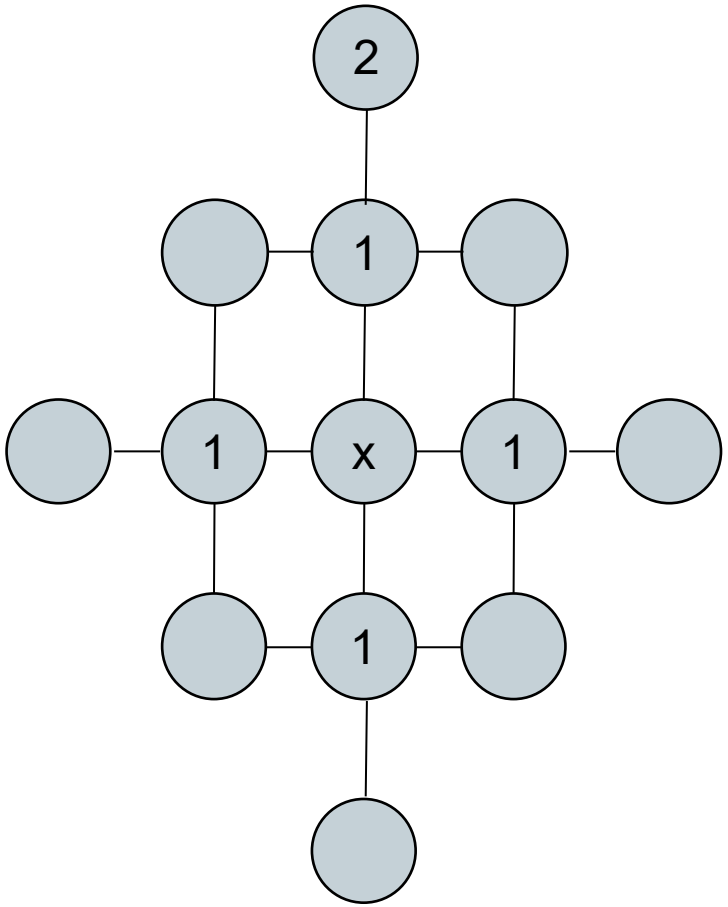
Busca em Profundidade

			5	4	5			
			4	3	4			
			3	2	3			
			2	1	2			
			1	X	1			
				1				

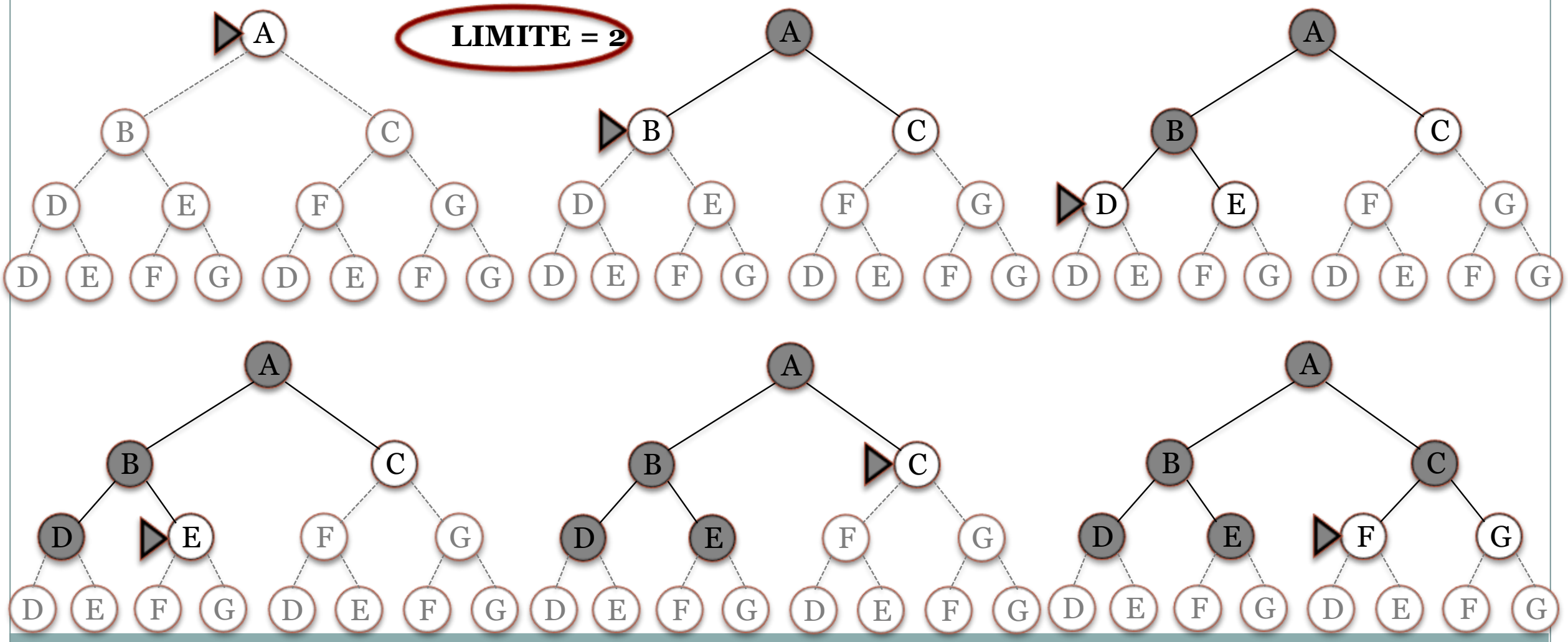


Busca em Profundidade

			5	4	5			
			4	3	4			
			3	2	3			
			2	1	2	3	4	5
			1	X	1	2	3	4
				1	2	3	4	5



Busca Profundidade Iterativa



Estratégias de Busca

- Busca Informada ou Heurística
  - Busca pela melhor escolha (*Best-First*)
  - Busca "gulosa" (*greedy*)
  - Busca A*
  - Busca IDA (A* interativo)
  - Subida de encosta (*HillClimbing*)

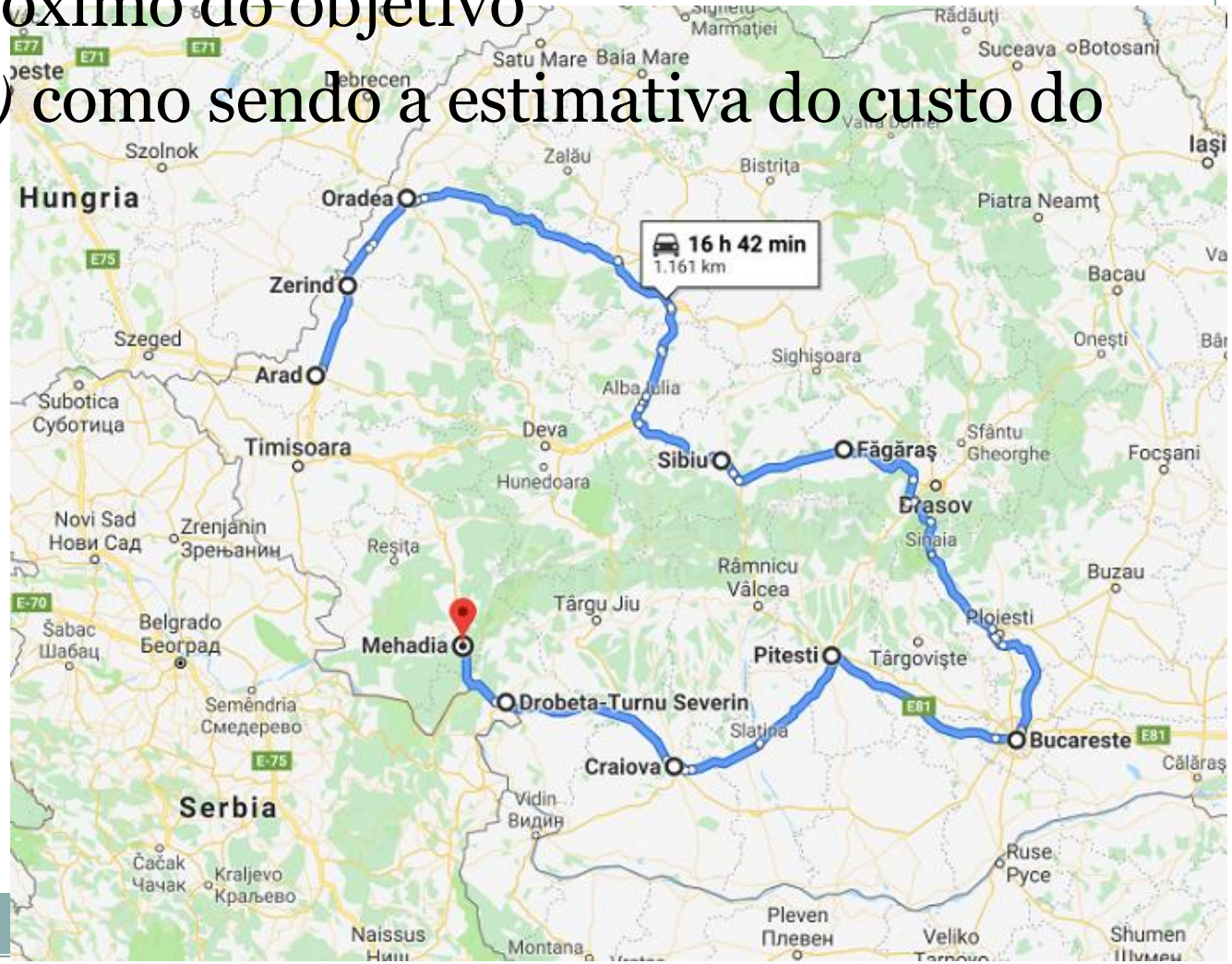
- Melhora a performance da busca através de uma função heurística *f(n)*, por exemplo:
  - minimização do custo para se atingir um objetivo;
  - estimativa do menor caminho de um estado *n* até um estado objetivo.

Busca Gulosa (*Greedy*)

- Expandir sempre o nodo mais próximo do objetivo
- Usa uma função de avaliação *h(n)* como sendo a estimativa do custo do nodo atual ao objetivo
- Exemplo: encontrar a rota da cidade de *Arad* até *Bucareste*

- Precisamos de uma...

HEURÍSTICA

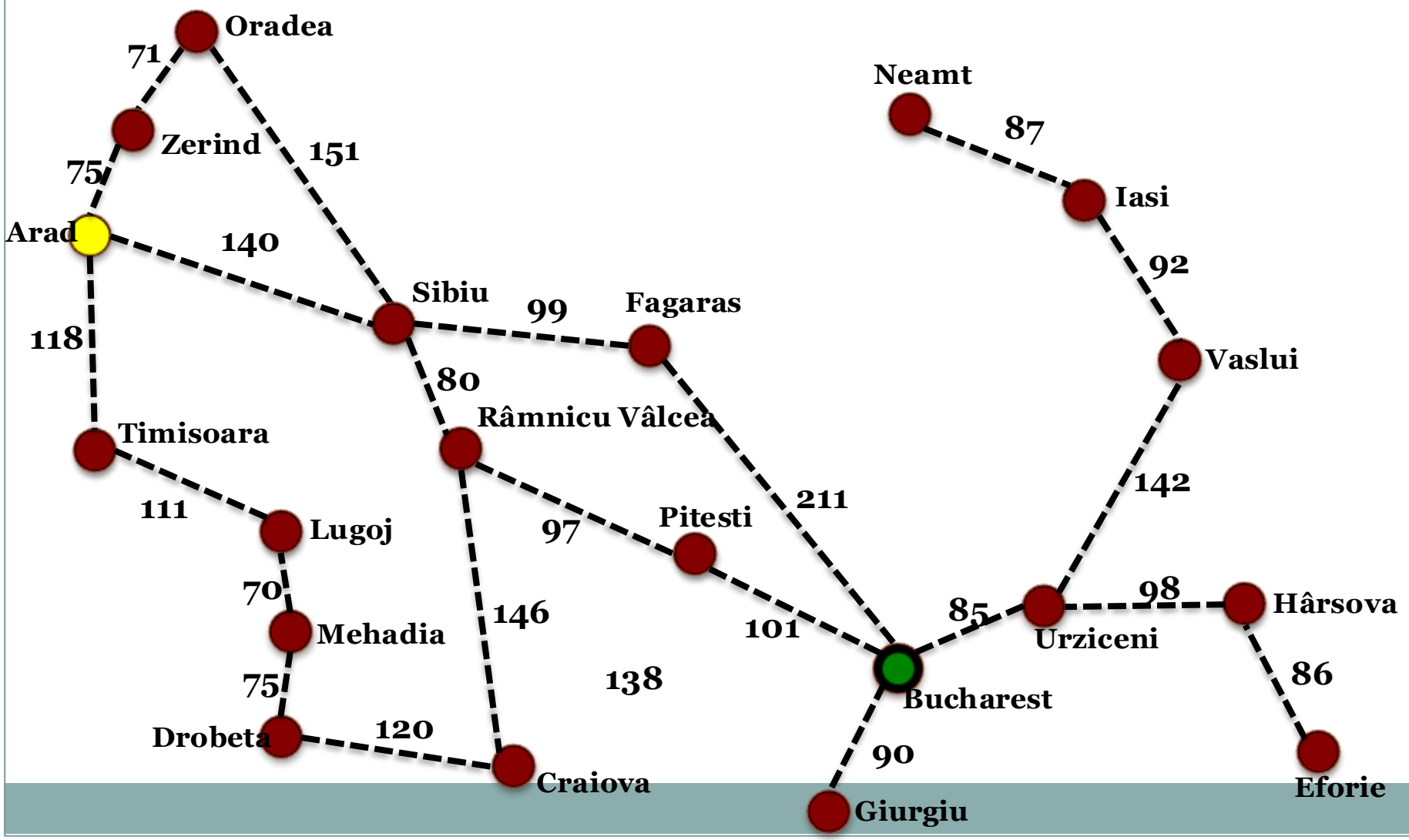


Busca Gulosa (*Greedy*)

- Expandir sempre o nodo mais próximo do objetivo
- Usa uma função de avaliação *h(n)* como sendo a estimativa do custo do nodo atual ao objetivo
- Exemplo: encontrar a rota da cidade de *Arad* até *Bucareste*

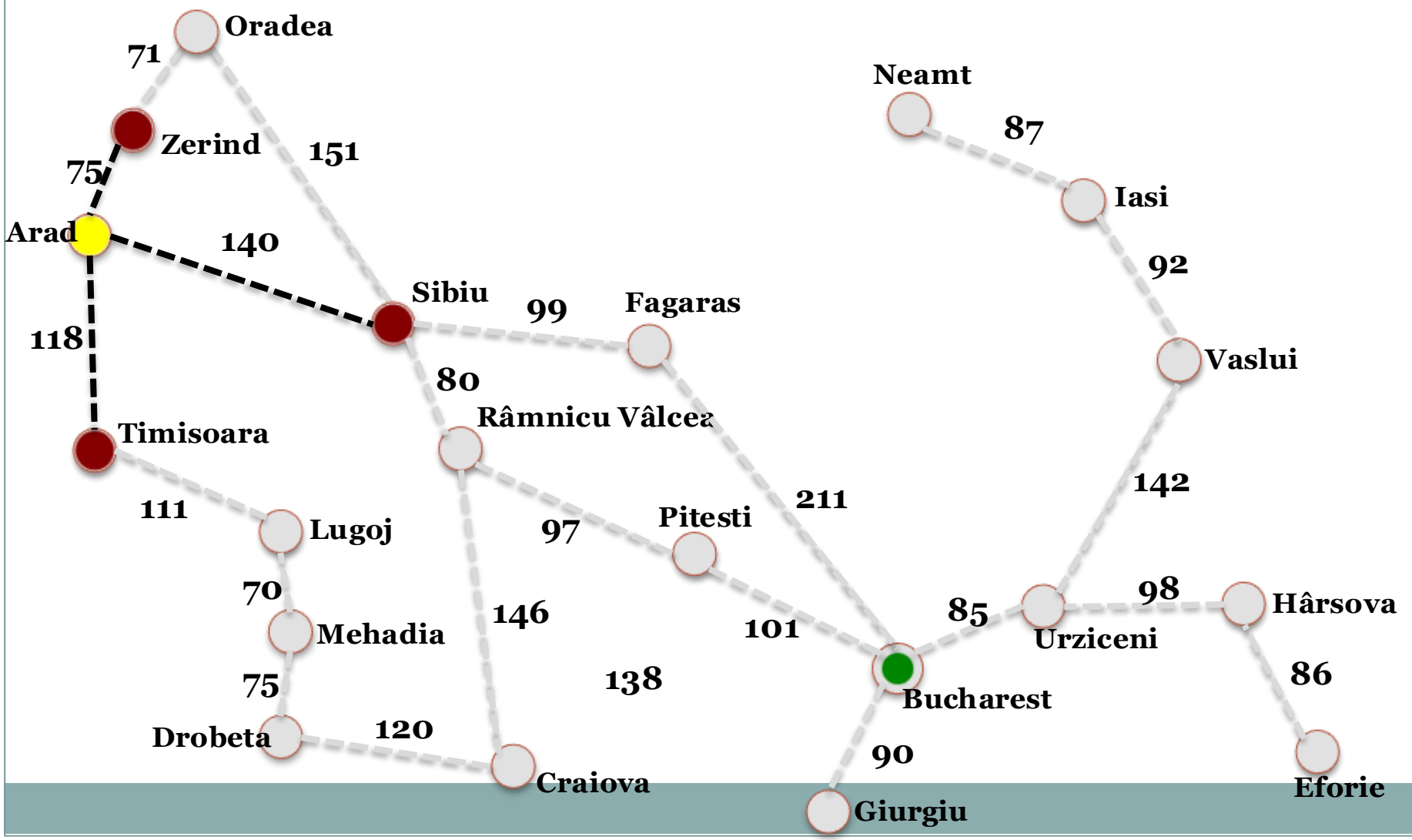
- Exemplo de heurística:
  - *h(n)* = distância em linha reta

Busca Gulosa (*Greedy*)



Distância em linha reta	
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Iasi	226
Lugoj	244
Hârsova	151
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Vaslui	199
Zerind	374
Urziceni	80

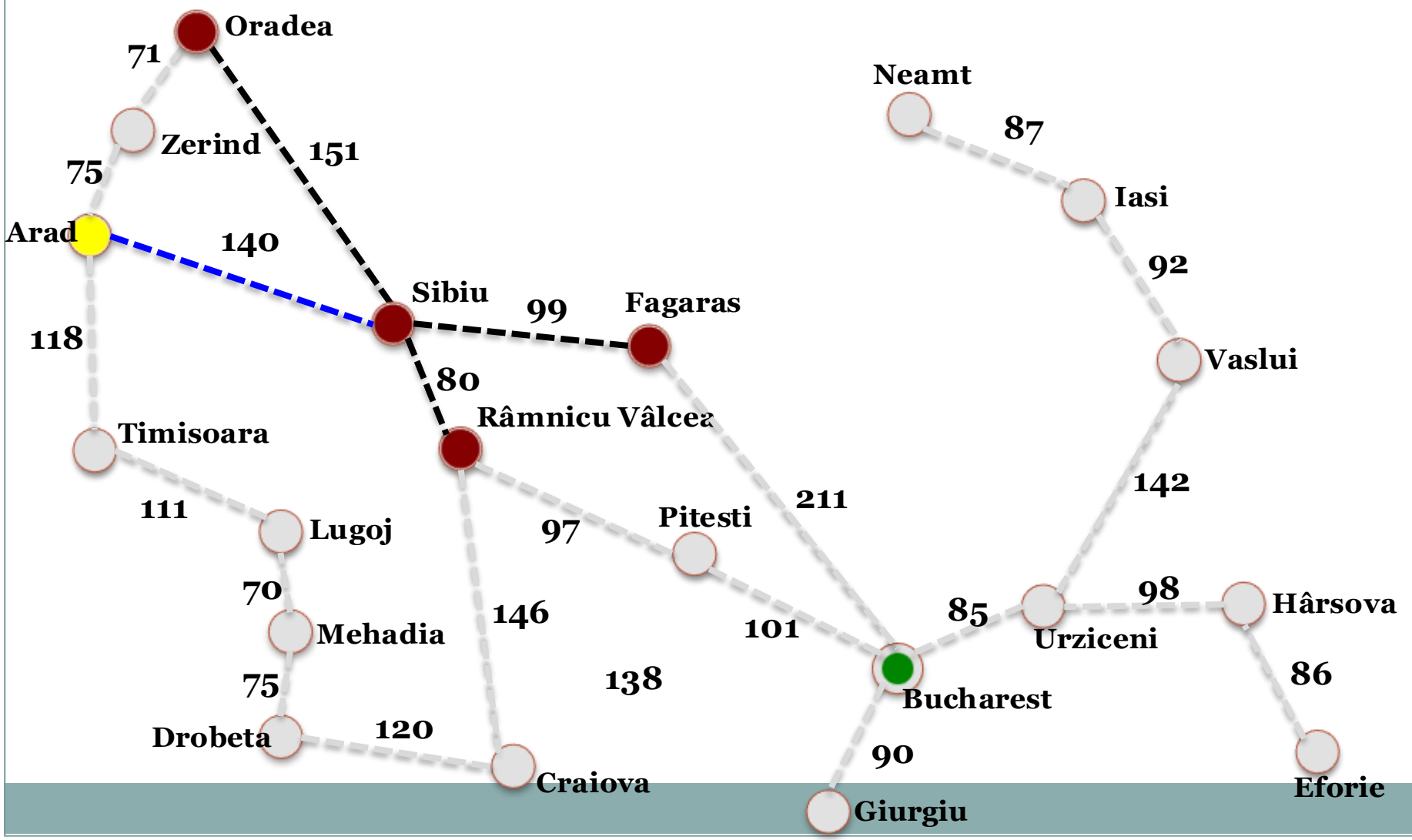
Busca Gulosa (*Greedy*)



Distância em linha reta

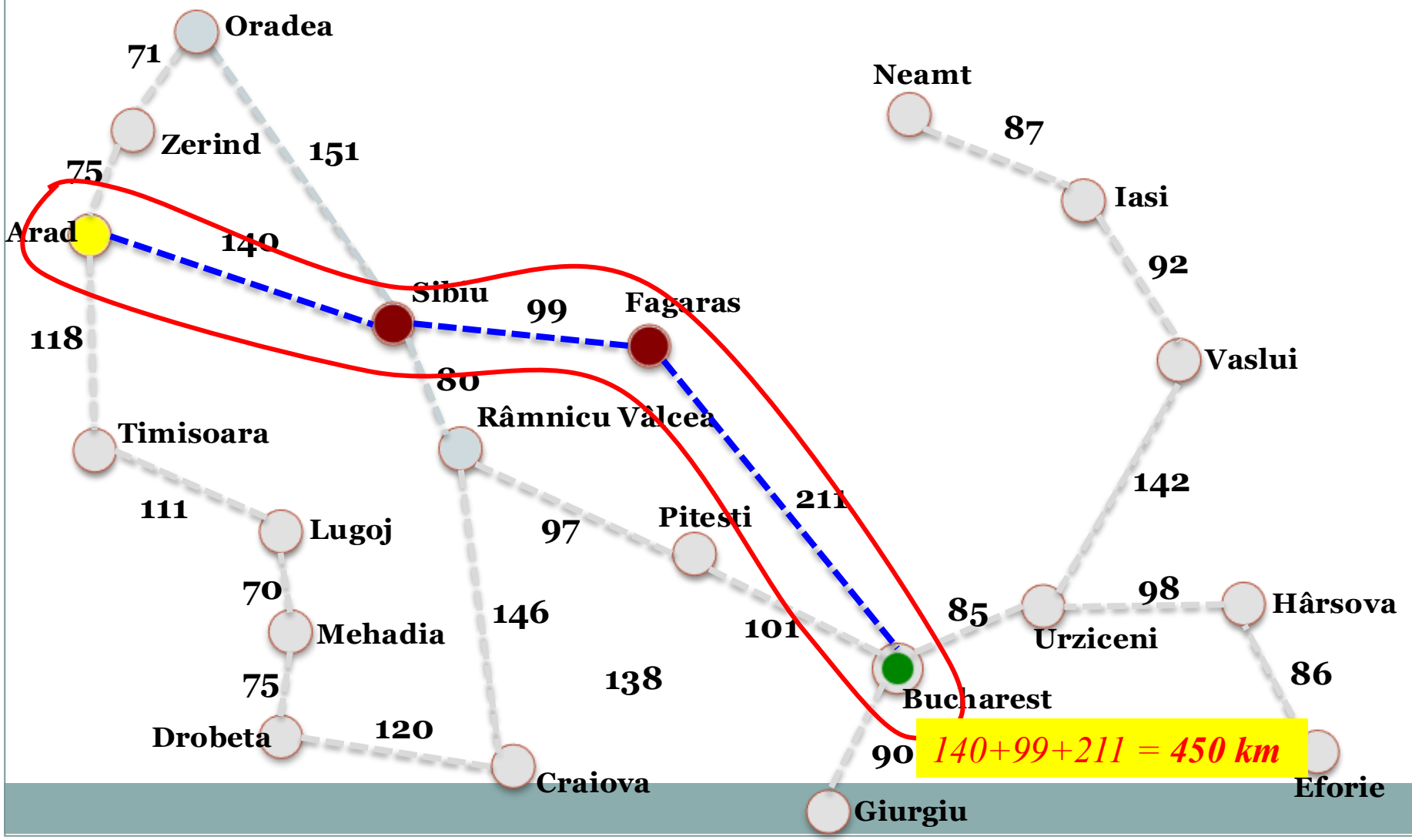
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Iasi	226
Lugoj	244
Hârsova	151
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Vaslui	199
Zerind	374
Urziceni	80

Busca Gulosa (*Greedy*)



Distância em linha reta	
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Iasi	226
Lugoj	244
Hârsova	151
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Vaslui	199
Zerind	374
Urziceni	80

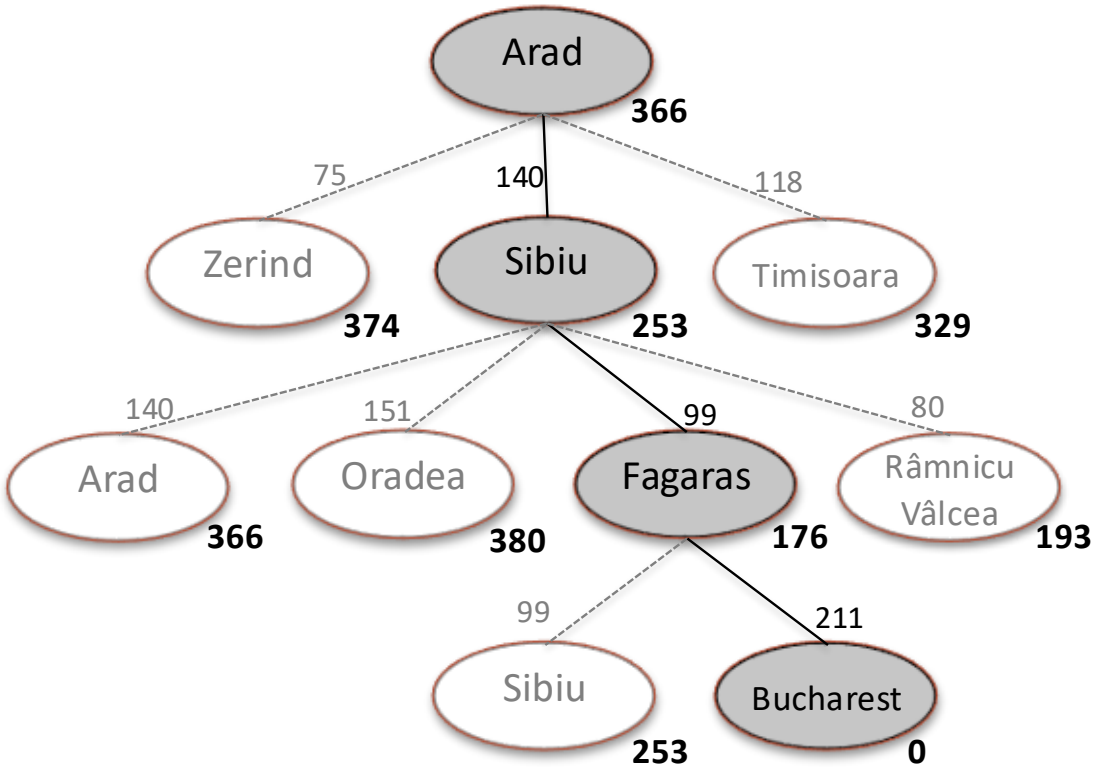
Busca Gulosa (*Greedy*)



Distância em linha reta

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Iasi	226
Lugoj	244
Hârsova	151
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Vaslui	199
Zerind	374
Urziceni	80

Busca Gulosa - Algoritmo



Custo Total:

f(n) = 140+99+211 = ***450 km***

h(n)	
Arad	366
Bucharest	0
...	
Fagaras	176
...	
Sibiu	253
...	

GET-BEST-NODE()
Obtém o melhor valor desta tabela

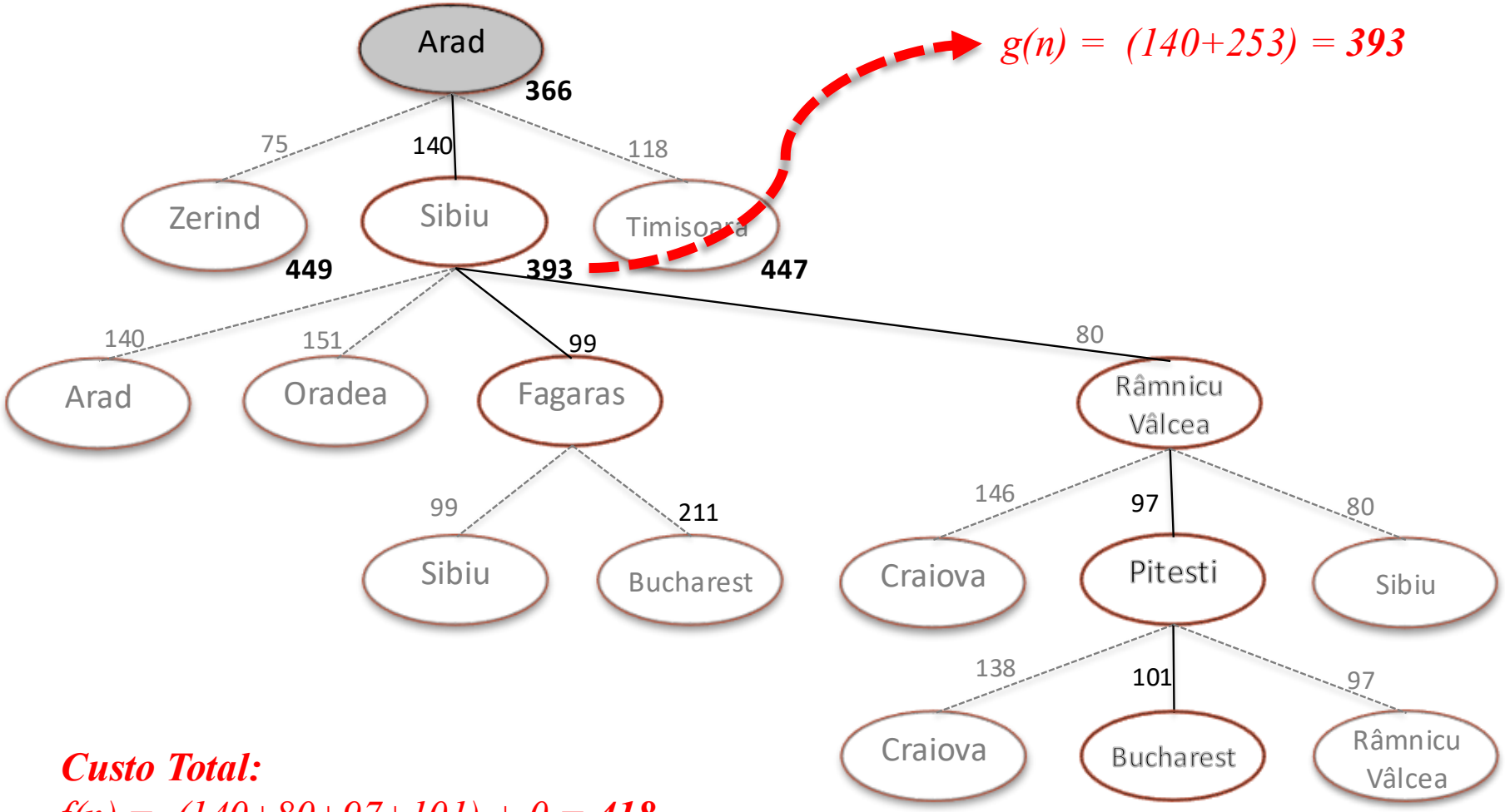
BREATH-FIRST(estadoInicial, sucesso)

nodos ← MAKE-QUEUE(estadoInicial)
REPITA
  SE nodos é vazio **ENTÃO RETORNA** falha
  nodo ← **GET-BEST_NODE(nodos)**
  SE GOAL-TEST(nodo)=sucesso **ENTÃO RETORNA** nodo
  novos-nodos ← EXPAND(nodo)
  nodos ← ADD-QUEUE(nodos, novos-nodos)
FIM

Busca A*

- Tenta minimizar o custo total da solução combinando:
  - *Busca Gulosa:* econômica, porém não é completa nem ótima
  - *Busca de Custo Uniforme:* ineficiente, porém completa e ótima
- Função de avaliação:
  - $f(n) = g(n) + h(n)$
  - $g(n)$ = distância de n ao nó inicial
  - $h(n)$ = distância estimada de n ao nó final
  - A* expande o nó de menor valor de f na fronteira do espaço de estados

Busca A*



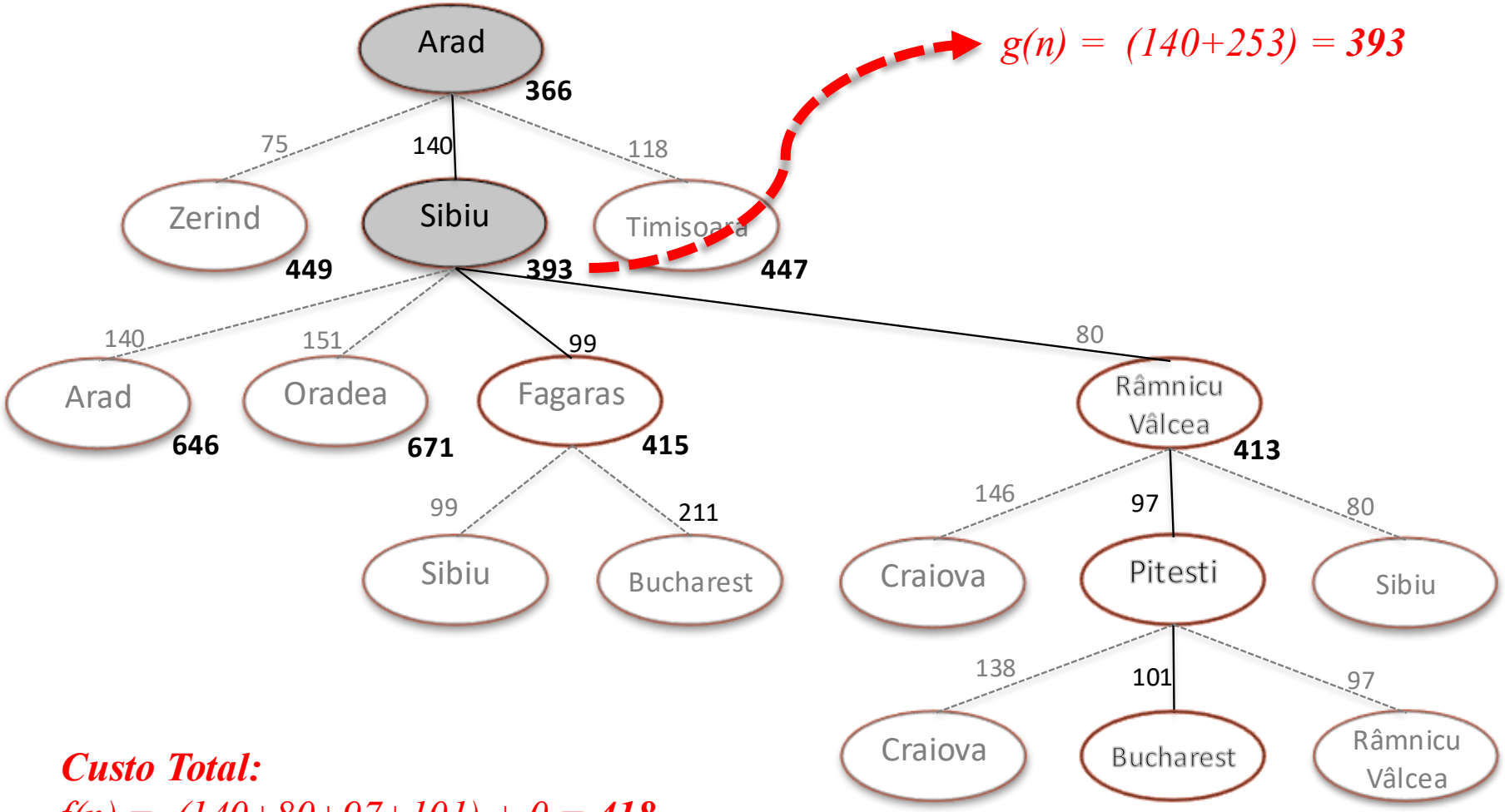
$g(n) = (140+253) = \mathbf{393}$

h(n)	
Bucharest	0
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Zerind	374

Custo Total:

$f(n) = (140+80+97+101) + 0 = \mathbf{418}$

Busca A*



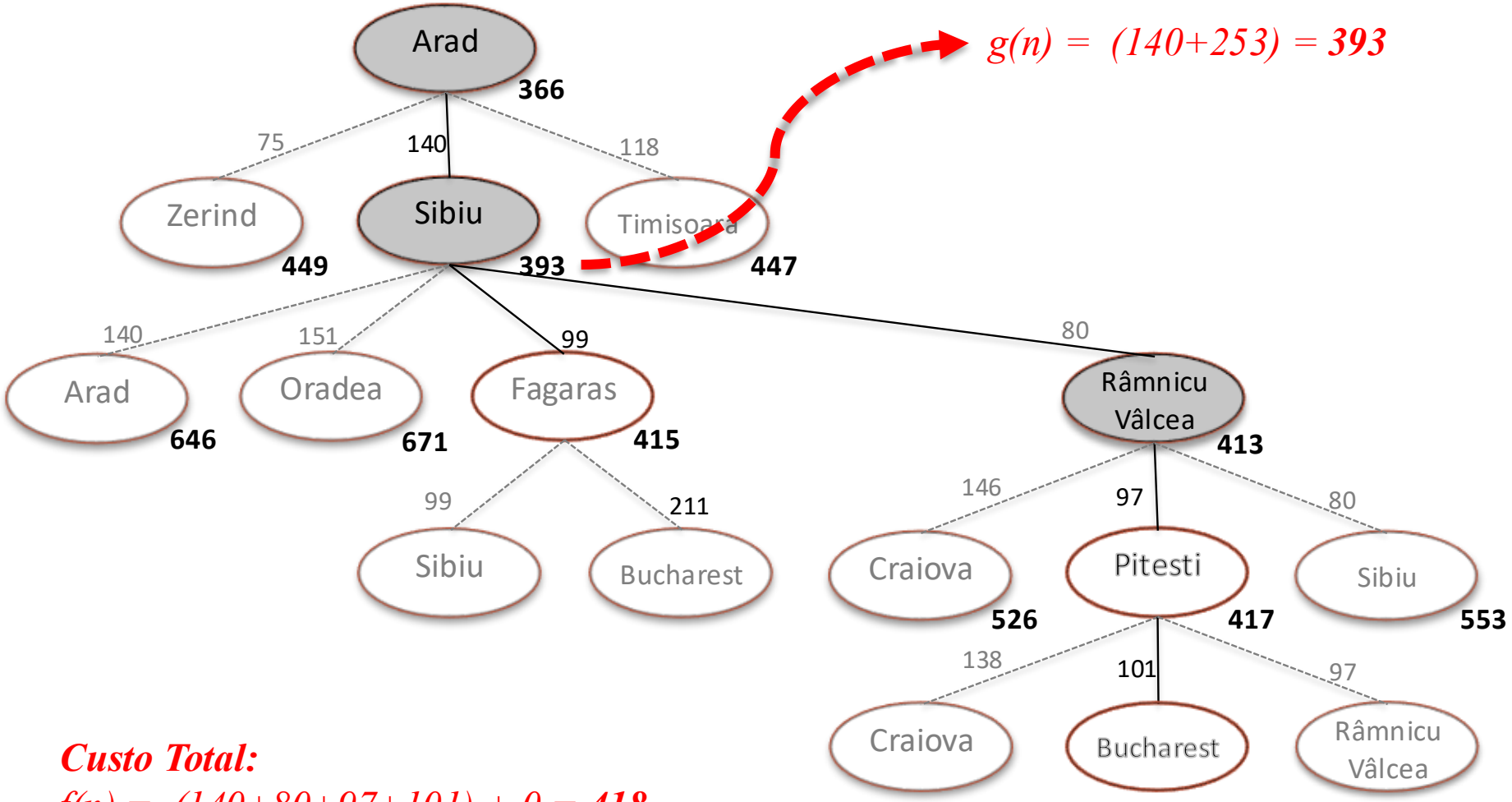
$g(n) = (140+253) = \mathbf{393}$

h(n)	
Bucharest	0
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Zerind	374

Custo Total:

$f(n) = (140+80+97+101) + 0 = \mathbf{418}$

Busca A*



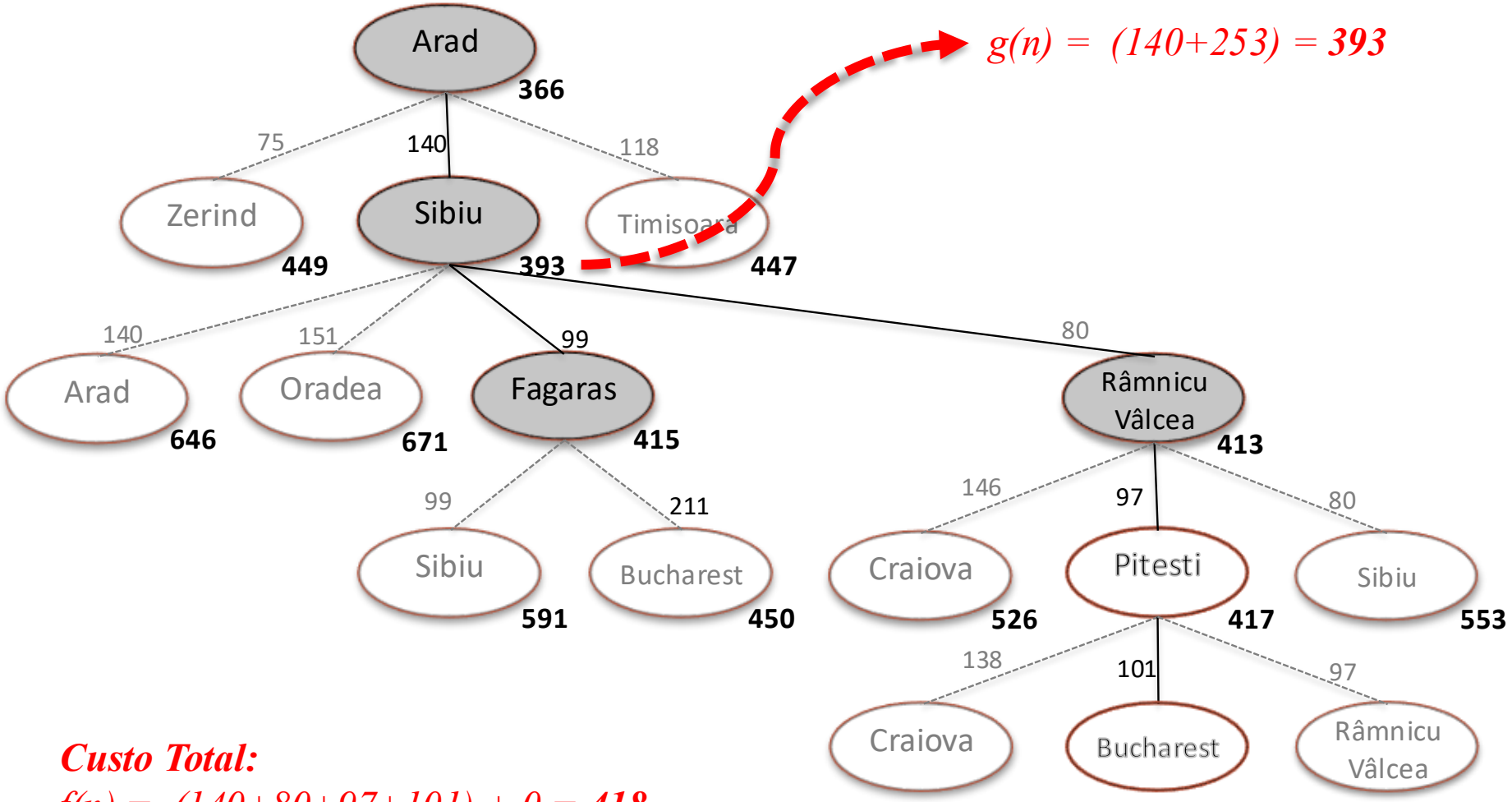
$g(n) = (140+253) = \mathbf{393}$

h(n)	
Bucharest	0
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Zerind	374

Custo Total:

$f(n) = (140+80+97+101) + 0 = \mathbf{418}$

Busca A*



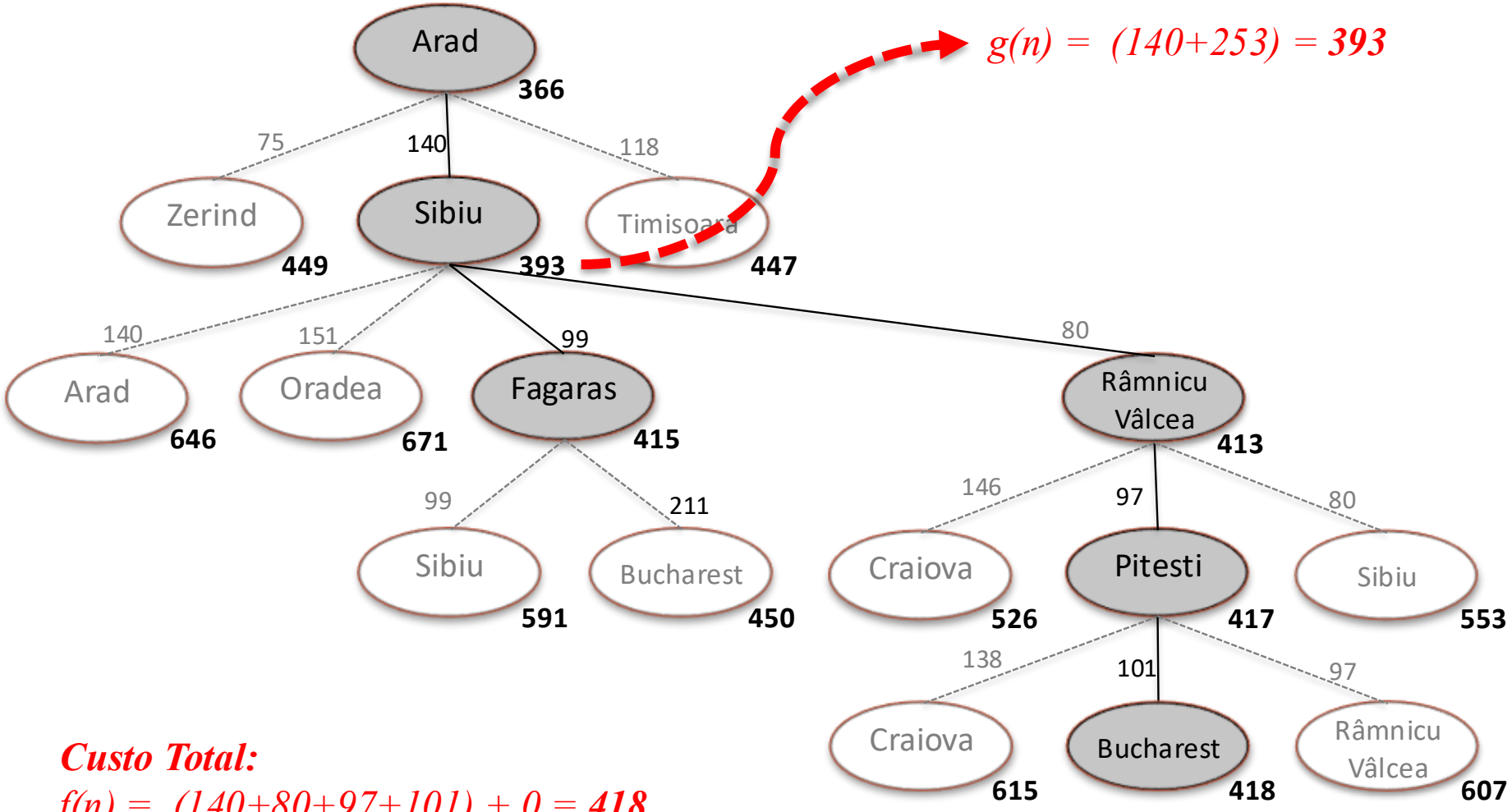
$g(n) = (140+253) = \mathbf{393}$

h(n)	
Bucharest	0
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Zerind	374

Custo Total:

$f(n) = (140+80+97+101) + 0 = \mathbf{418}$

Busca A*



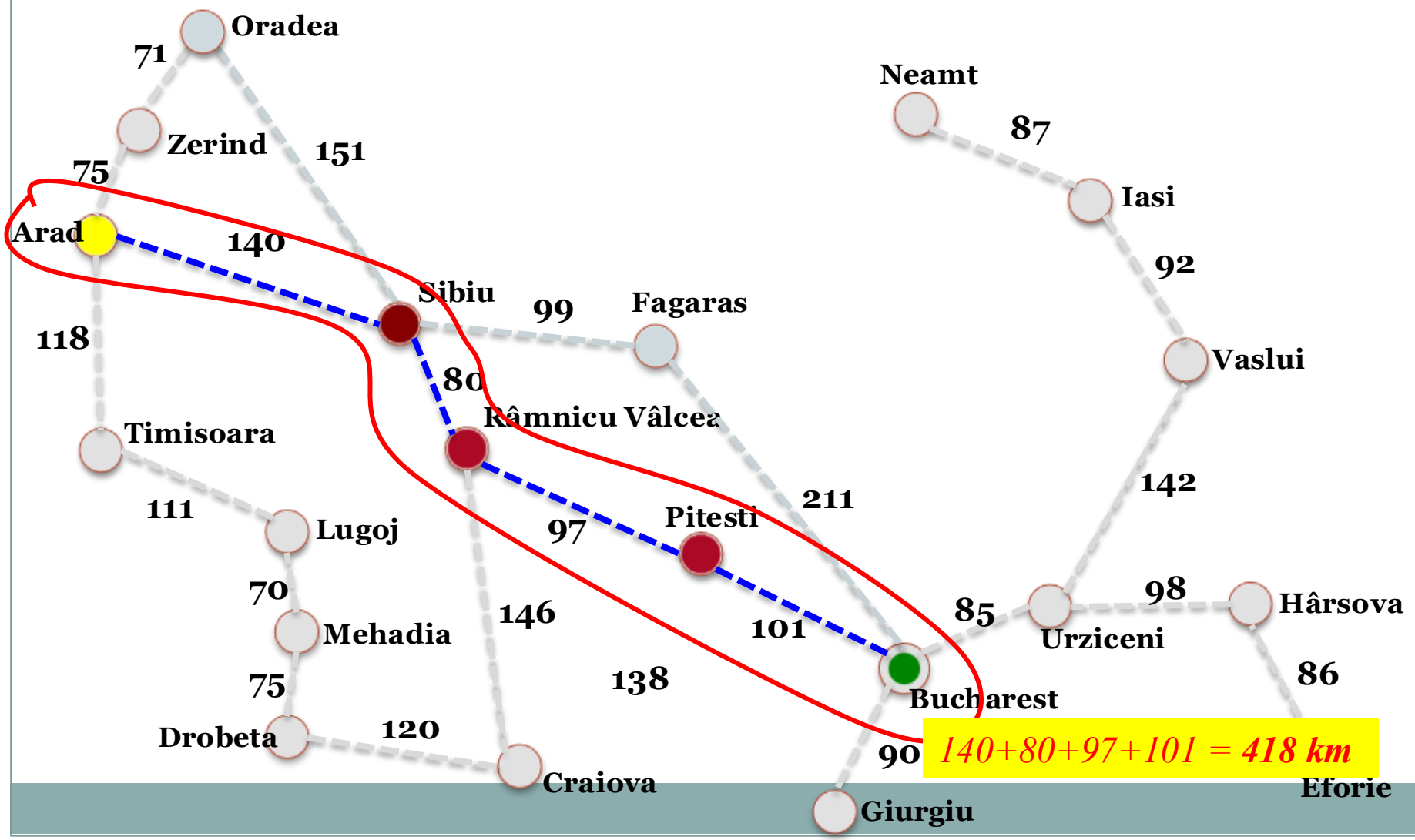
$g(n) = (140+253) = \mathbf{393}$

h(n)	
Bucharest	0
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Zerind	374

Custo Total:

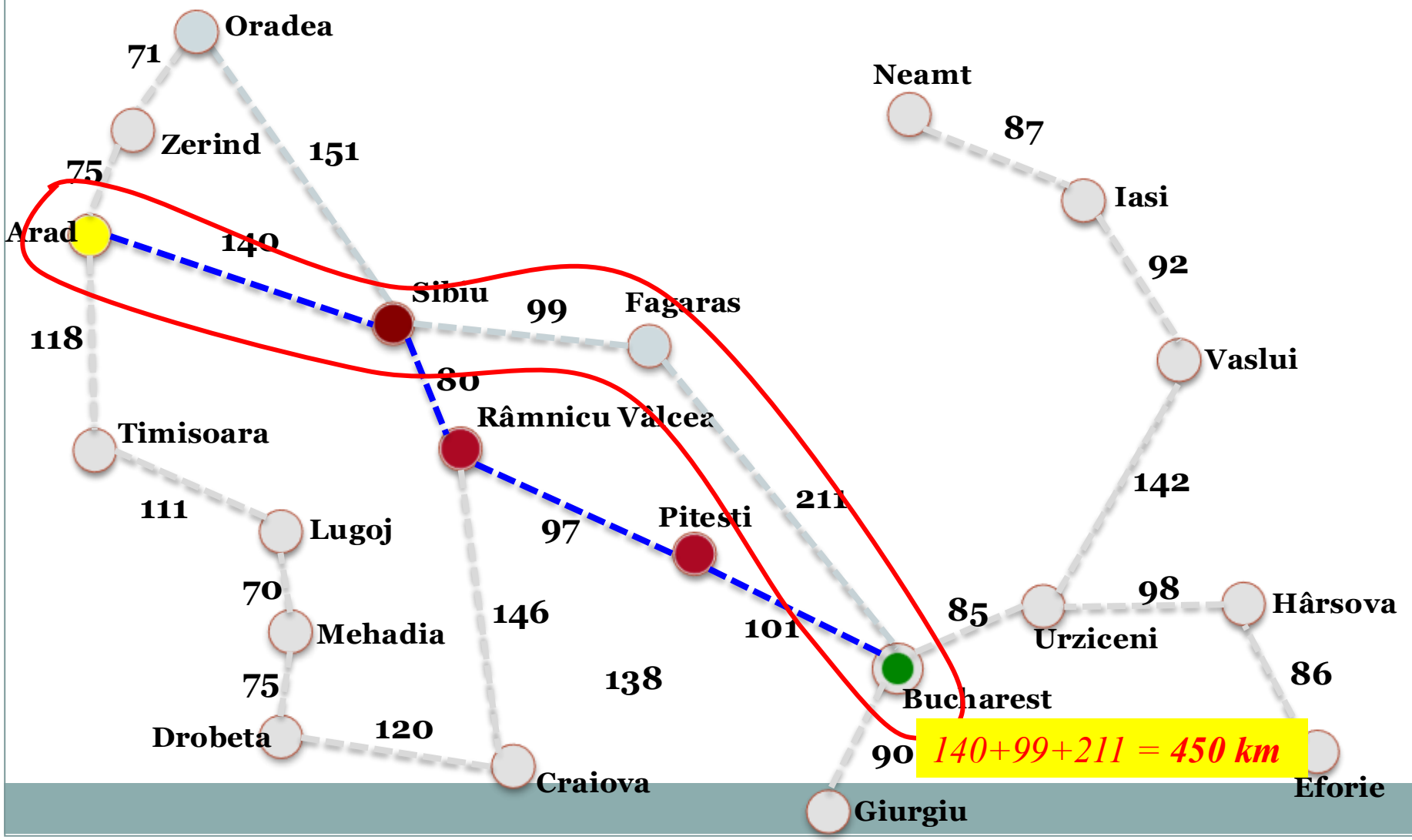
$f(n) = (140+80+97+101) + 0 = \mathbf{418}$

Busca A*



Distância em linha reta	
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Iasi	226
Lugoj	244
Hârsova	151
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Vaslui	199
Zerind	374
Urziceni	80

Busca Gulosa (*Greedy*)



Distância em linha reta

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Iasi	226
Lugoj	244
Hârsova	151
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Râmnicu Vâlcea	193
Sibiu	253
Timisoara	329
Vaslui	199
Zerind	374
Urziceni	80

Avaliação de Algoritmos de Busca

- Critérios na escolha de uma estratégia:
  - **Completude**
    - o algoritmo oferece a garantia de encontrar uma solução quando ela existir, e reportar corretamente a falha quando não houver solução?
  - **Complexidade de Tempo**
    - quanto tempo ele leva para encontrar uma solução? Isso pode ser medido em segundos ou, de modo mais abstrato, pelo número de estados e ações consideradas.
  - **Complexidade de Espaço**
    - quanta memória é necessária para realizar a busca?
  - **Qualidade** (*optimality*) **da solução**
    - a estratégia encontra a melhor solução quando existem diferentes soluções?

Avaliação de Algoritmos de Busca

Algoritmo	Completo?	Ótimo?	Tempo	Espaço
BFS	Sim	Sim[1]		
DFS	Não	Não		
IDS	Sim	Sim[1]		
UCS	Sim[2]	Sim[2]		
A*	Sim[2]	Sim[3]		

b : Fator de ramificação (número máximo de sucessores de qualquer nó).
d : Profundidade da solução mais rasa (objetivo mais próximo da raiz).
m : Profundidade máxima do espaço de busca (pode ser infinito).
l : Limite de profundidade escolhido pelo usuário.
C^* : Custo da solução ótima.
ε : Custo mínimo de cada etapa (deve ser maior que zero).

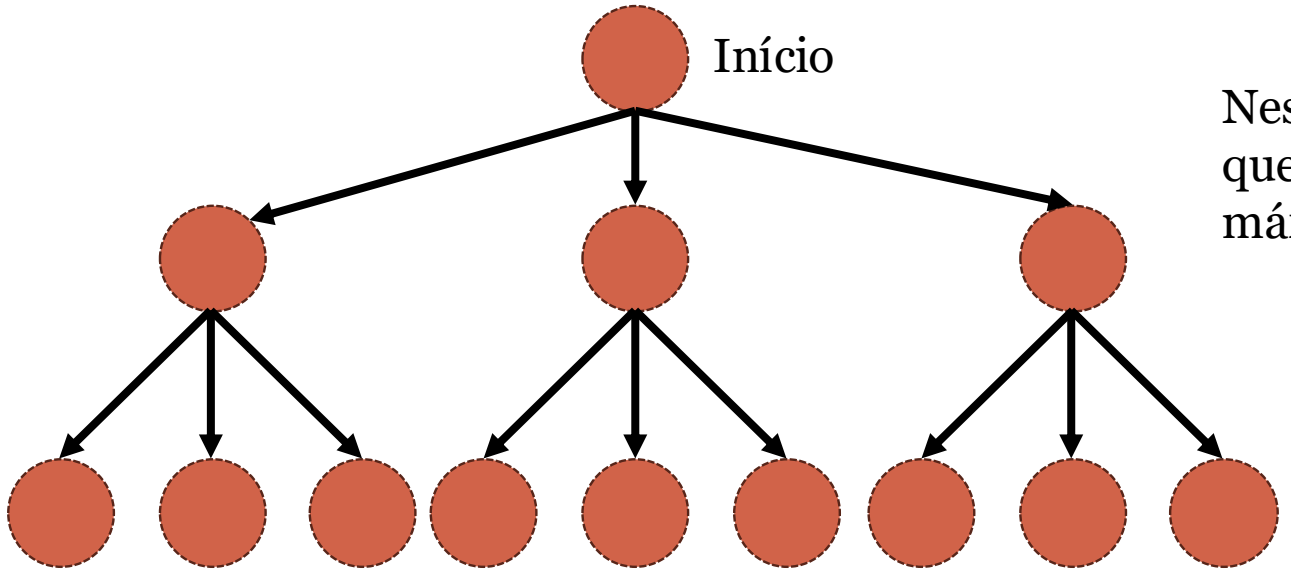
Avaliação de Algoritmos de Busca

Algoritmo	Completo?	Ótimo?	Tempo	Espaço
BFS	Sim	Sim[1]		
DFS	Não	Não		
IDS	Sim	Sim[1]		
UCS	Sim[2]	Sim[2]		
A*	Sim[2]	Sim[3]		

[1] Se o fator de ramificação for finito.
[2] Se o custo de cada passo for $\geq \varepsilon$ (evita caminhos infinitos com custos decrescentes).
[3] Se o custo de cada passo for idêntico (ex: todos os passos custam 1).
[4] Assume que ambos os lados usam busca em largura e que o teste de objetivo é eficiente.

Caracterização de problemas de busca

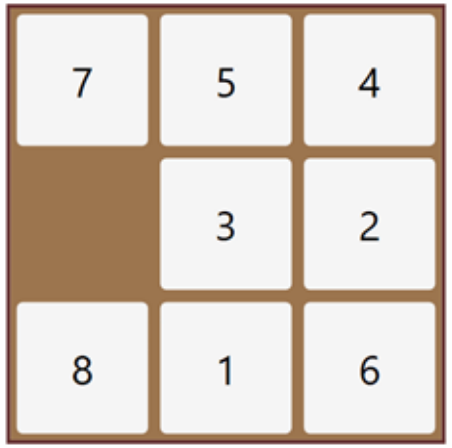
- *b*: fator de ramificação
  - No pior caso, o número máximo de sucessores que cada nodo pode ter



Neste caso, b=3, uma vez que cada nodo tem no máximo 3 sucessores.

Caracterização de problemas de busca

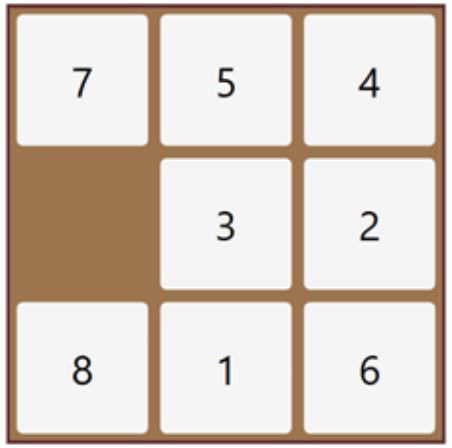
- *b*: fator de ramificação
  - No pior caso, o número máximo de sucessores que cada nodo pode ter



Qual é o fator de ramificação do *8-puzzle*?

Caracterização de problemas de busca

- *b*: fator de ramificação
  - No pior caso, o número máximo de sucessores que cada nodo pode ter

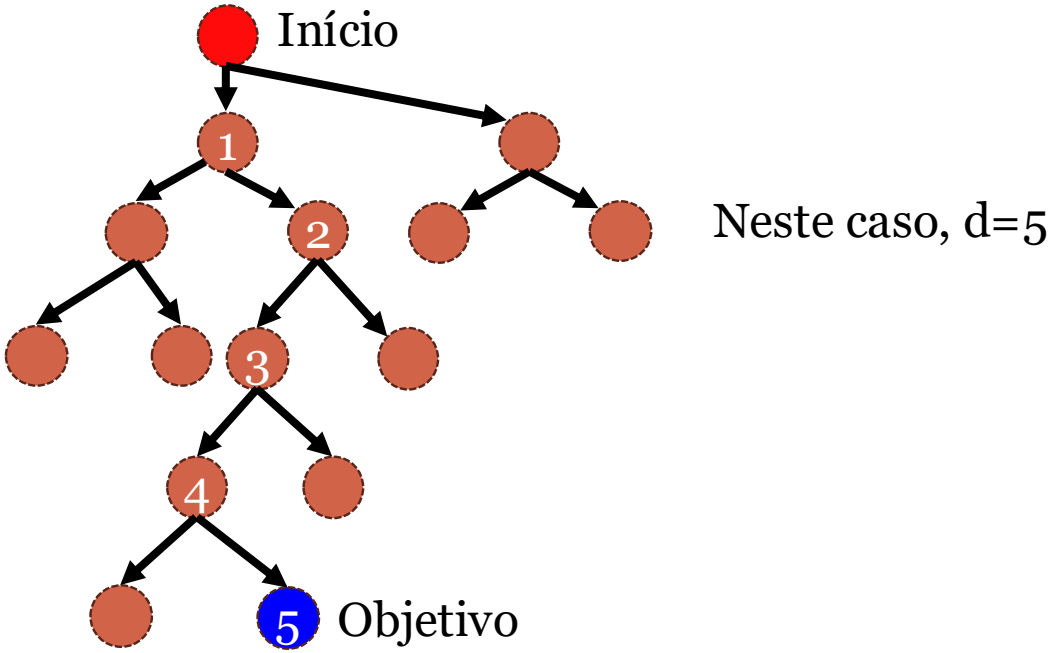


Qual é o fator de ramificação do *8-puzzle*?

b=4, uma vez que no máximo, o espaço vazio pode ser movido para uma das 4 posições adjacentes na horizontal ou vertical

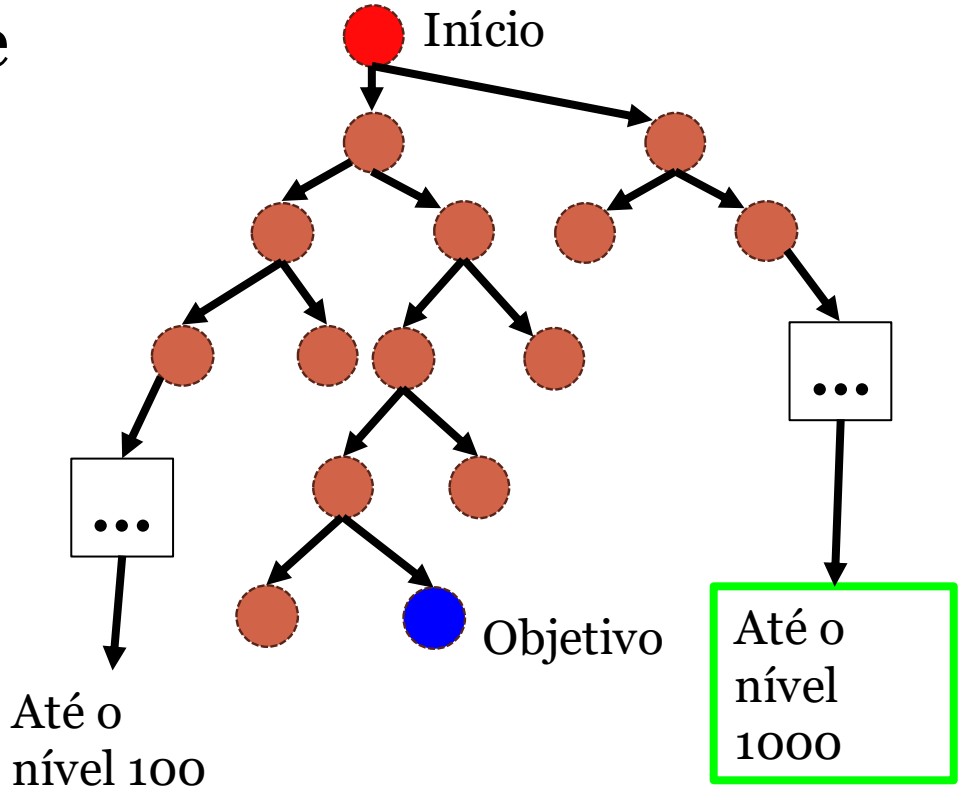
Caracterização de problemas de busca

- *b*: fator de ramificação
- *d*: profundidade da solução mais rasa



Caracterização de problemas de busca

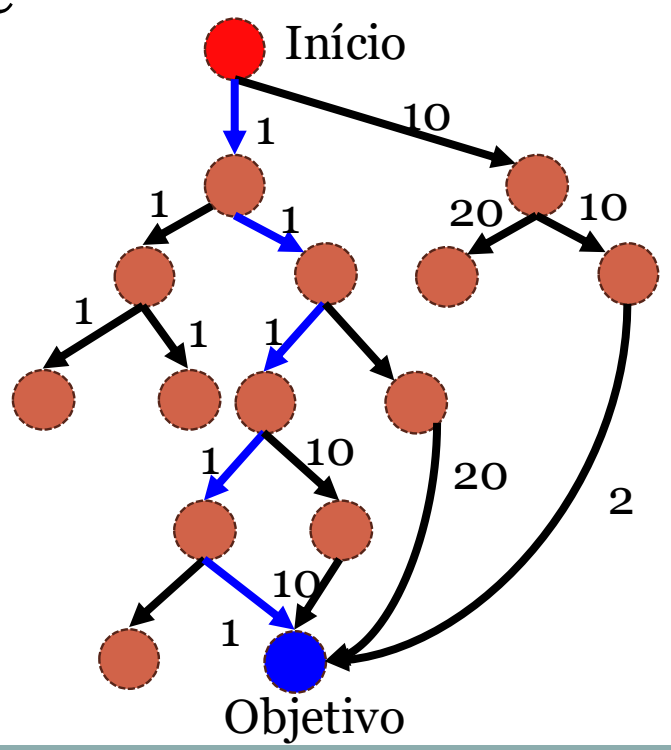
- *b*: fator de ramificação
- *d*: profundidade da solução mais rasa
- *m*: profundidade máxima de qualquer caminho



Neste caso, m=1000

Caracterização de problemas de busca

- *b*: fator de ramificação
- *d*: profundidade da solução mais rasa
- *m*: profundidade máxima de qualquer caminho
- *C*: custo de uma solução
  - C^*: custo da solução ótima



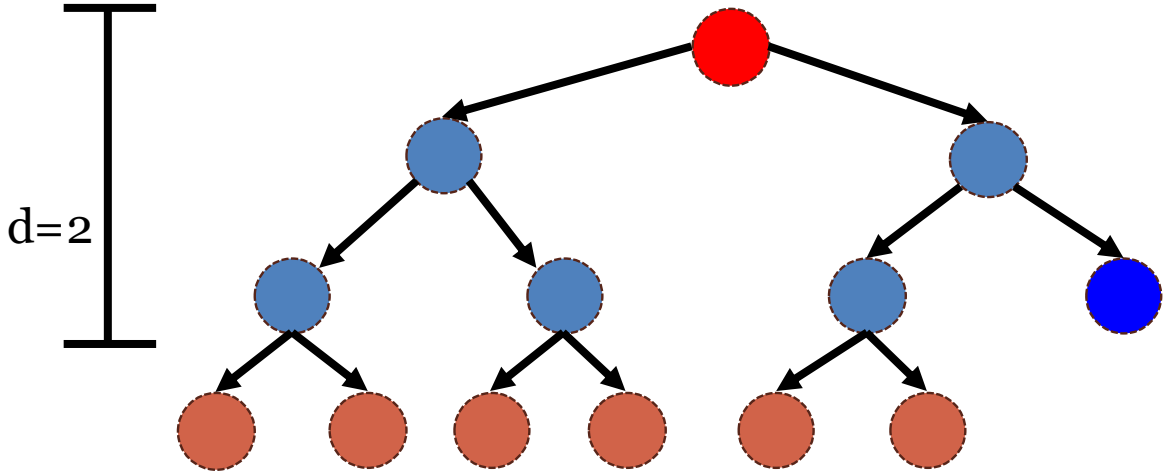
Neste caso, c*=5
Embora existam outras soluções com outros custos

Busca em largura (BFS) – complexidade de tempo

- *b*: fator de ramificação
- *d*: profundidade da solução mais rasa
- *m*: profundidade máxima de qualquer caminho
- *C*: custo de uma solução
  - C^*: custo da solução ótima

Busca em largura (BFS)

- **Complexidade de tempo**: $O(b^{d+1})$
- **Complexidade de espaço**:



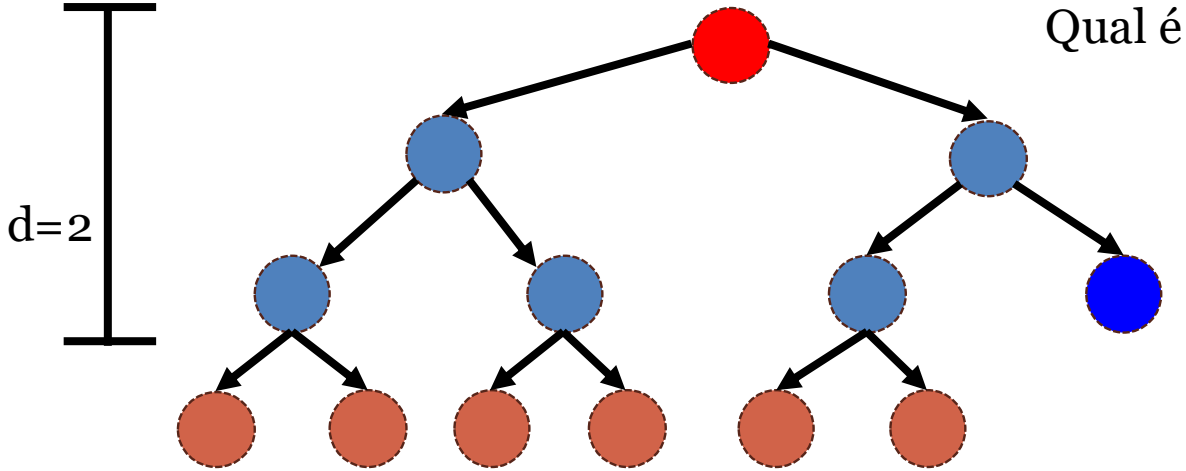
Pode cair para $O(b^d)$ se testarmos se o nodo é o objetivo assim que ele é gerado em vez de testar quando ele é visitado

Busca em largura (BFS) – complexidade de tempo

- b: fator de ramificação
- d: profundidade da solução mais rasa
- m: profundidade máxima de qualquer caminho
- C: custo de uma solução
  - C^*: custo da solução ótima

Busca em largura (BFS)

- **Complexidade de tempo**: $O(b^{d+1})$
- **Complexidade de espaço**:



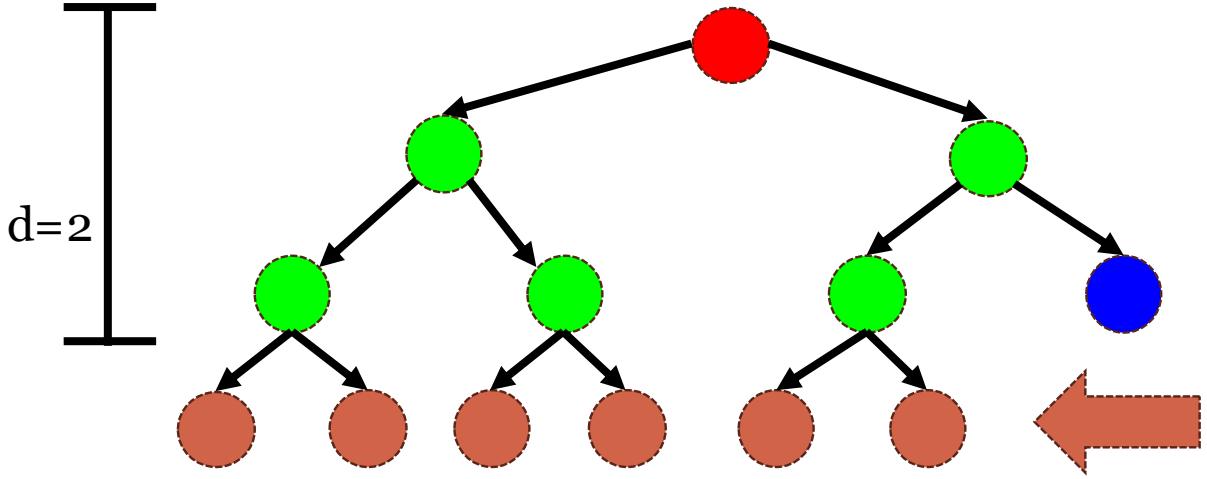
Qual é o custo de espaço?

Busca em largura (BFS) – complexidade de espaço

- b: fator de ramificação
- d: profundidade da solução mais rasa
- m: profundidade máxima de qualquer caminho
- C: custo de uma solução
  - C^*: custo da solução ótima

Busca em largura (BFS)

- **Complexidade de tempo**: $O(b^{d+1})$
- **Complexidade de espaço**:



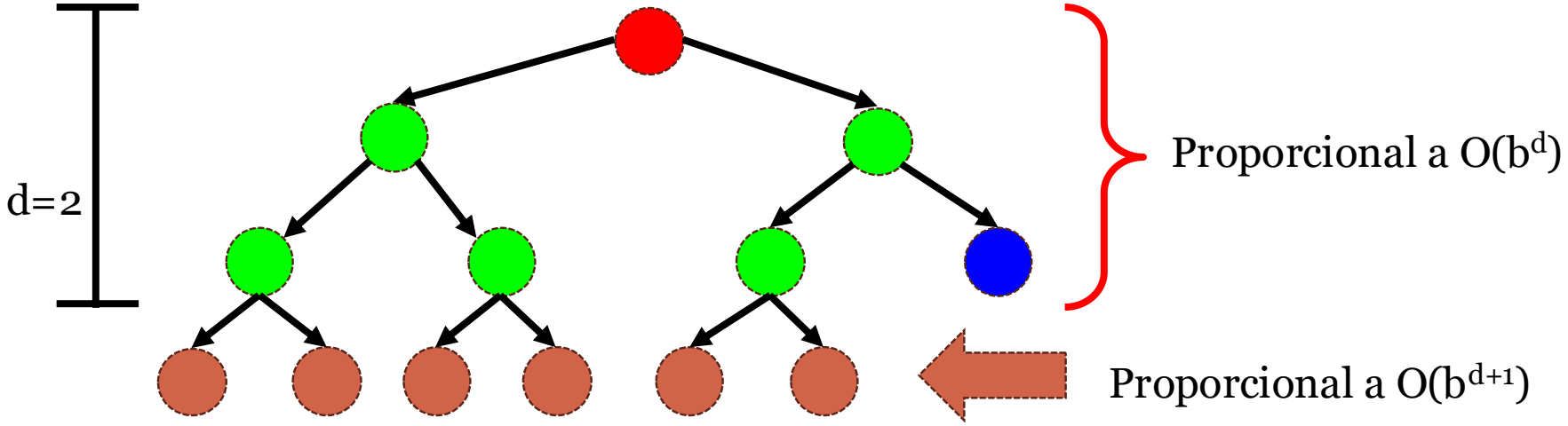
Fronteira: proporcional a $O(b^{d+1})$

Busca em largura (BFS) – complexidade de espaço

- b: fator de ramificação
- d: profundidade da solução mais rasa
- m: profundidade máxima de qualquer caminho
- C: custo de uma solução
  - C^*: custo da solução ótima

Busca em largura (BFS)

- **Complexidade de tempo**: $O(b^{d+1})$
- **Complexidade de espaço**: $O(b^{d+1})$

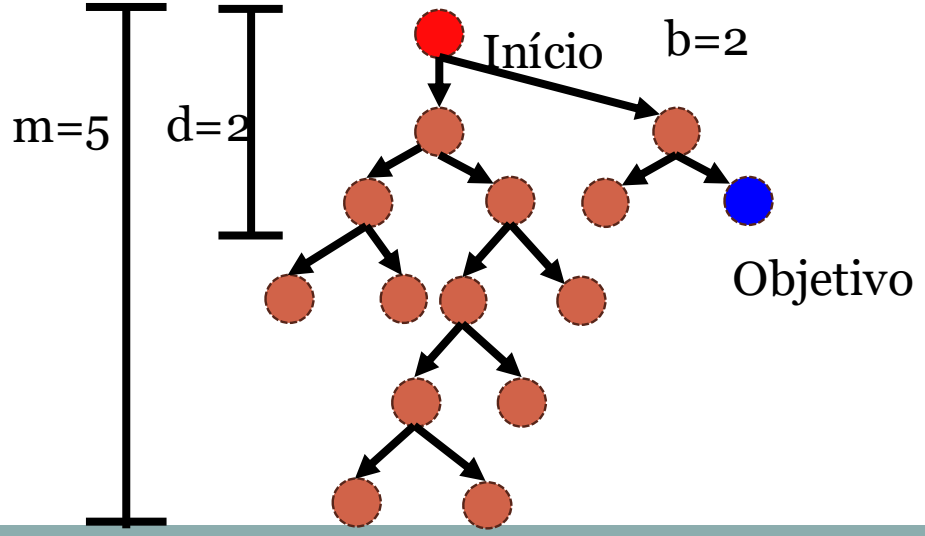


Busca em profundidade (DFS)

- b: fator de ramificação
- d: profundidade da solução mais rasa
- m: profundidade máxima de qualquer caminho
- C: custo de uma solução
  - C^*: custo da solução ótima

Busca em profundidade (DFS)

- **Complexidade de tempo**: $O(b^m)$
- **Complexidade de espaço**: X+O(bm)



O que domina a complexidade de espaço é m.

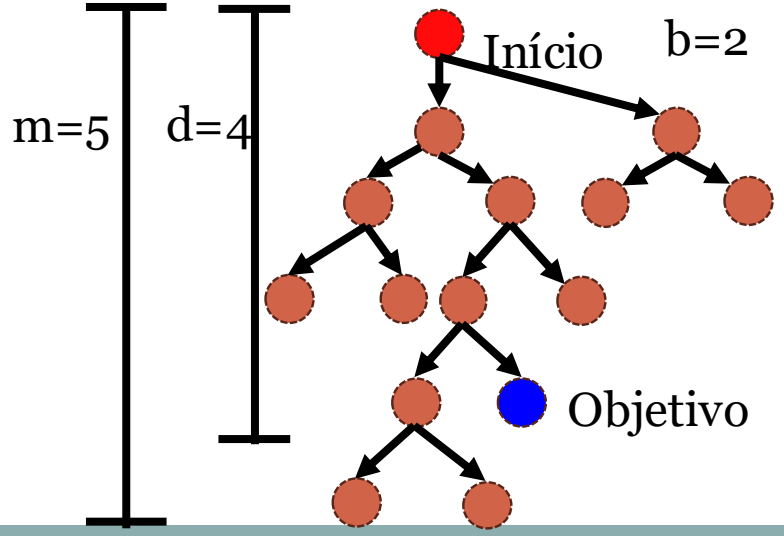
Considerando que b sucessores são gerados a cada nodo, e a profundidade máxima de busca é m, o algoritmo precisa armazenar no máximo b nodos a cada nível m, portanto a complexidade é proporcional a O(bm)

Busca com aprofundamento iterativo (IDS)

- b: fator de ramificação
- d: profundidade da solução mais rasa
- m: profundidade máxima de qualquer caminho
- C: custo de uma solução
  - C^*: custo da solução ótima

Busca em com aprofundamento iterativo (IDS)

- **Complexidade de tempo**:
- **Complexidade de espaço**:



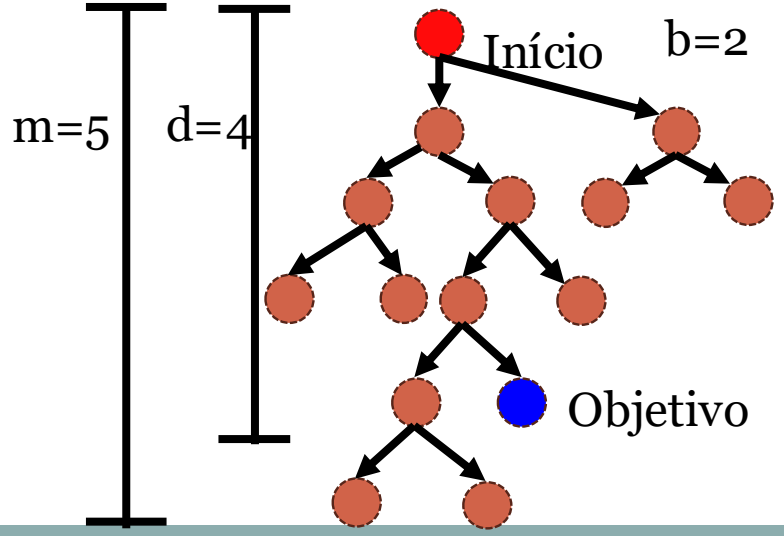
O que domina a complexidade de tempo do IDS?

Busca com aprofundamento iterativo (IDS)

- b: fator de ramificação
- d: profundidade da solução mais rasa
- m: profundidade máxima de qualquer caminho
- C: custo de uma solução
  - C^*: custo da solução ótima

Busca em com aprofundamento iterativo (IDS)

- **Complexidade de tempo**:
- **Complexidade de espaço**:



O que domina a complexidade de tempo do IDS?

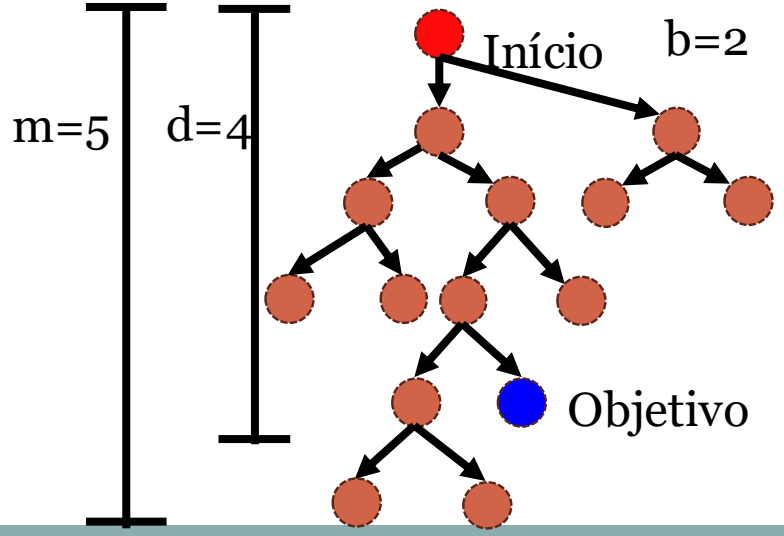
A profundidade da solução mais rasa!

Busca com aprofundamento iterativo (IDS)

- *b*: fator de ramificação
- *d*: profundidade da solução mais rasa
- *m*: profundidade máxima de qualquer caminho
- *C*: custo de uma solução
  - C^*: custo da solução ótima

Busca em com aprofundamento iterativo (IDS)

- **Complexidade de tempo**: $O(b^d)$
- **Complexidade de espaço**:



O nós da profundidade *d* são gerados apenas uma vez, os nós da profundidade *d-1* são gerados 2 vezes,..., e os filhos da raiz são gerados *d* vezes.

Então, o tempo é dado por:

$$d*b+(d-1)*b^2+...+1*b^d$$

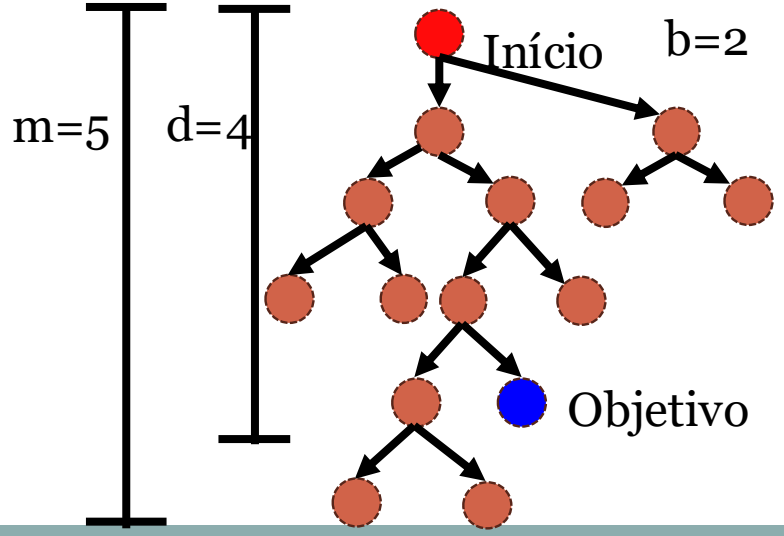
A complexidade de tempo é proporcional a $O(b^d)$.

Busca com aprofundamento iterativo (IDS)

- *b*: fator de ramificação
- *d*: profundidade da solução mais rasa
- *m*: profundidade máxima de qualquer caminho
- *C*: custo de uma solução
  - C^*: custo da solução ótima

Busca em com aprofundamento iterativo (IDS)

- **Complexidade de tempo**: $O(b^d)$
- **Complexidade de espaço**: X+O(bd)



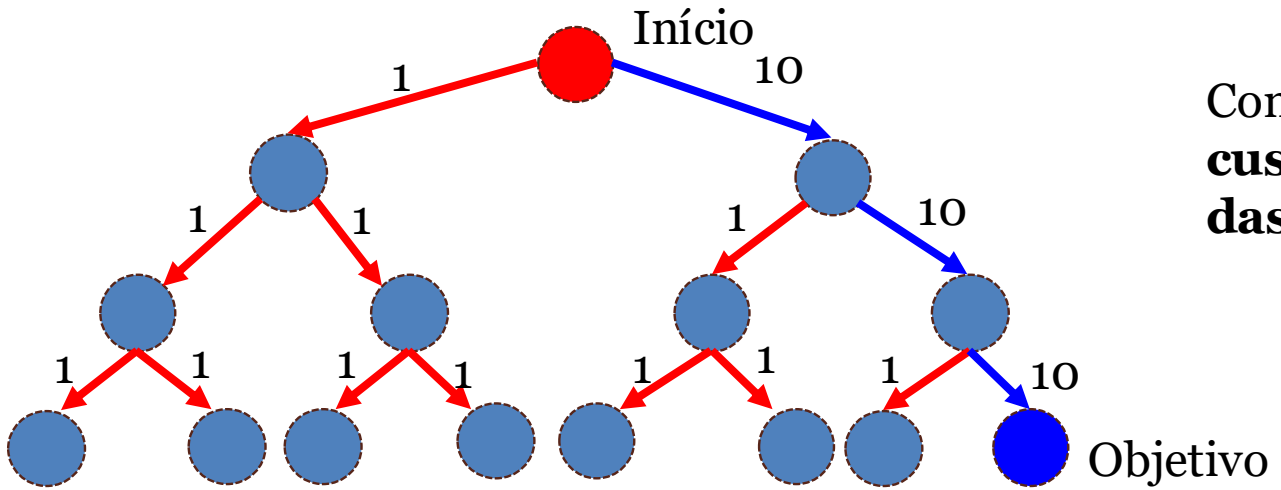
Na d-ésima busca em profundidade limitada realizada, o algoritmo precisa armazenar *b* nodos em cada nível, logo, o espaço necessário para armazenar todos os nodos é proporcional a O(bd)

Busca de custo uniforme (UCS)

- b: fator de ramificação
- d: profundidade da solução mais rasa
- m: profundidade máxima de qualquer caminho
- C: custo de uma solução
  - C^*: custo da solução ótima

Busca de custo uniforme (UCS)

- **Complexidade de tempo**:
- **Complexidade de espaço**:



Complexidade de tempo depende do **custo C^* do caminho ótimo** e o **custo das ações**!

Busca de custo uniforme (UCS)

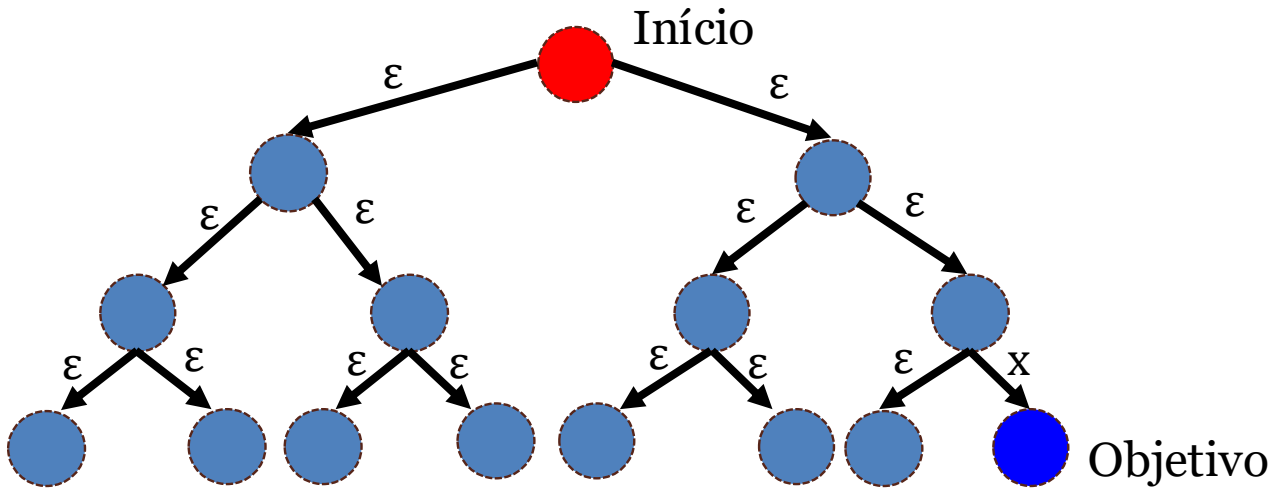
UCS sempre vai procurar **caminhos de menor custo antes**.

UCS expande todos os nós de custo menor que o caminho ótimo, com caminhos de profundidade máxima proporcional a 1 (da raiz) + $\lfloor C^*/\varepsilon \rfloor$.

A cada nível, cada nodo gera *b* sucessores.

Busca de custo uniforme (UCS)

- **Complexidade de tempo**: $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
- **Complexidade de espaço**: $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

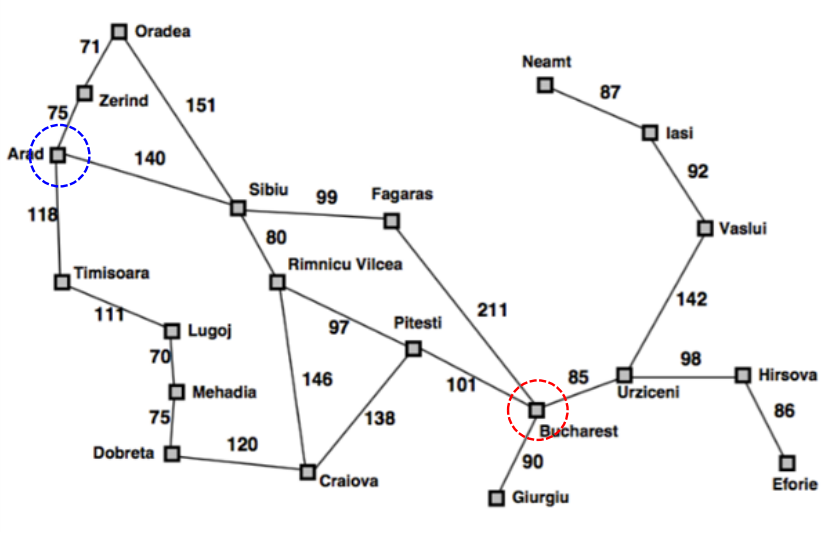


A*

- b: fator de ramificação
- d: profundidade da solução mais rasa
- m: profundidade máxima de qualquer caminho
- C: custo de uma solução
  - C^*: custo da solução ótima

A*

- **Complexidade de tempo**:
- **Complexidade de espaço**:



O que controla a complexidade de tempo e espaço do A*?

A heurística!

A*

PIOR heurística admissível e consistente?

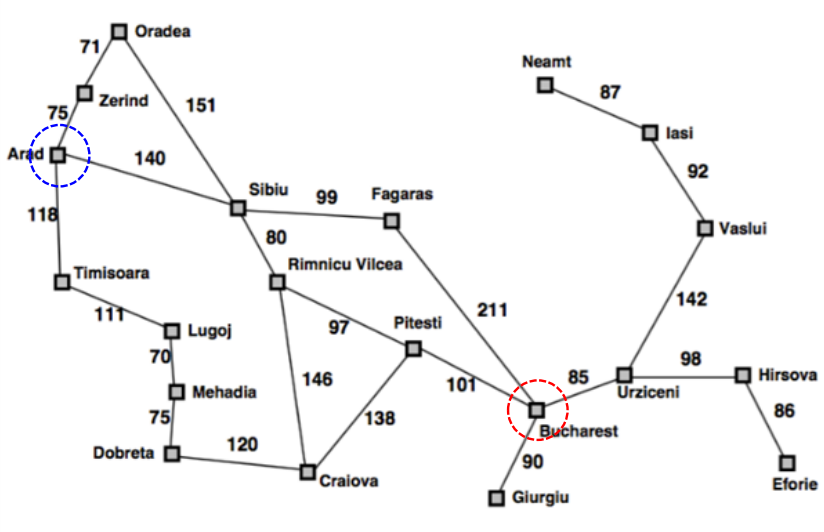
A mais otimista de todas!

h(v) = 0

A*

- **Complexidade de tempo**: $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
- **Complexidade de espaço**: $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

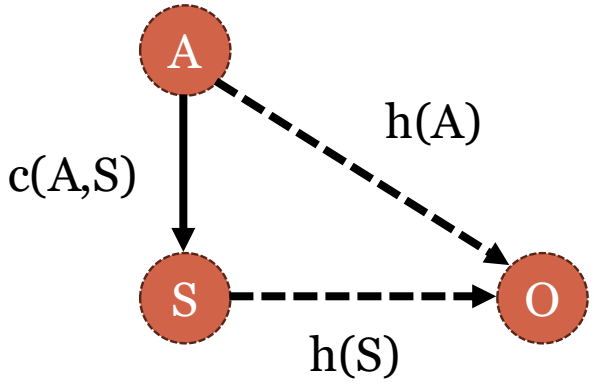
= UCS



MELHOR heurística admissível e consistente?

A* - Heurística

- **Completude**: sempre encontra uma solução, se existir?
  - Sim, se custos das arestas forem positivos e cada nodo tem um número finito de sucessores.
- **Otimalidade**: sempre encontra solução de menor custo?
  - Sim, se custos das arestas sempre forem positivos e heurística for admissível e consistente
  - Admissível indica que a estimativa deve subestimar o custo real (otimismo).
  - Consistente significa satisfazer a desigualdade triangular.

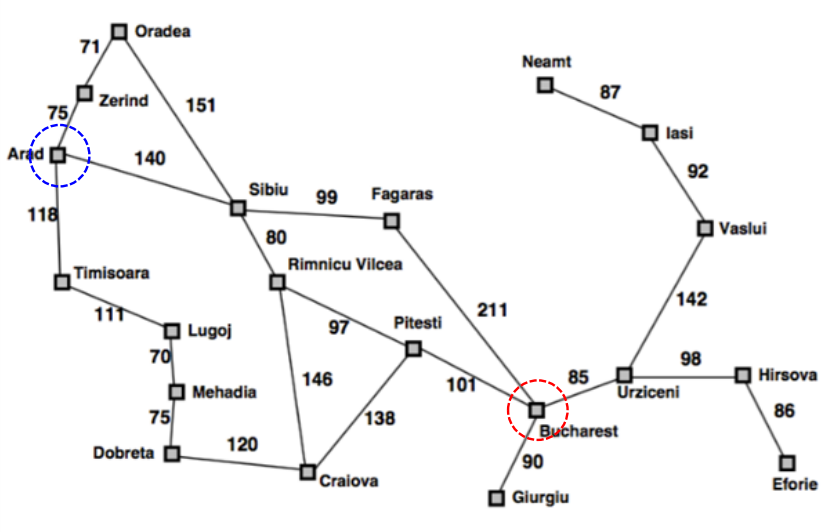


Uma heurística h(v) é consistente se, para todo nodo A, para cada sucessor S de A, é verdade que:
$h(A) \leq c(A,S) + h(s)$

A*

- Quando a heurística é perfeita, o A* percorre apenas o caminho ótimo, mas gera todos os *b* sucessores de cada nodo no caminho. Assim, a complexidade de tempo é proporcional a $O(bp)$, onde p é o tamanho do caminho ótimo.

- **Complexidade de tempo**: Melhor heurística: O(bp) e Pior heurística: $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
- **Complexidade de espaço**: Melhor heurística: O(bp) e Pior heurística: $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$



Características dos algoritmos de busca

Algoritmo	Completo?	Ótimo?	Tempo	Espaço
BFS	Sim	Sim[1]	$O(b^{d+1})$	$O(b^{d+1})$
DFS	Não	Não	$O(b^{m})$	$O(bm)$
IDS	Sim	Sim[1]	$O(b^{d})$	$O(bd)$
UCS	Sim[2]	Sim[2]	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$
A*	Sim[2]	Sim[3]	$O(bp)$ $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(bp)$ $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$

1. Em grafos de custo homogêneo
2. Se todos os custos são positivos
3. Se heurística é admissível e consistente